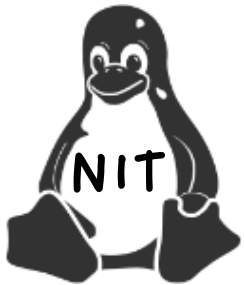


Welcome
Everyone



MASTERING
LINUX!

MODULE 05 - updated
(June 14th / June 20th 2026)

LINUX

File Operations

HISTORY COMMAND

#history

```
#ls -al
#cat .bash_history
```

What is the maximum no. of commands it stores?

#echo \$HISTFILESIZE

```
[tom@VM01 ~]$ echo $HISTFILESIZE
1000
```

```
[nitadmin2@VM01 ~]$ history
```

oldest

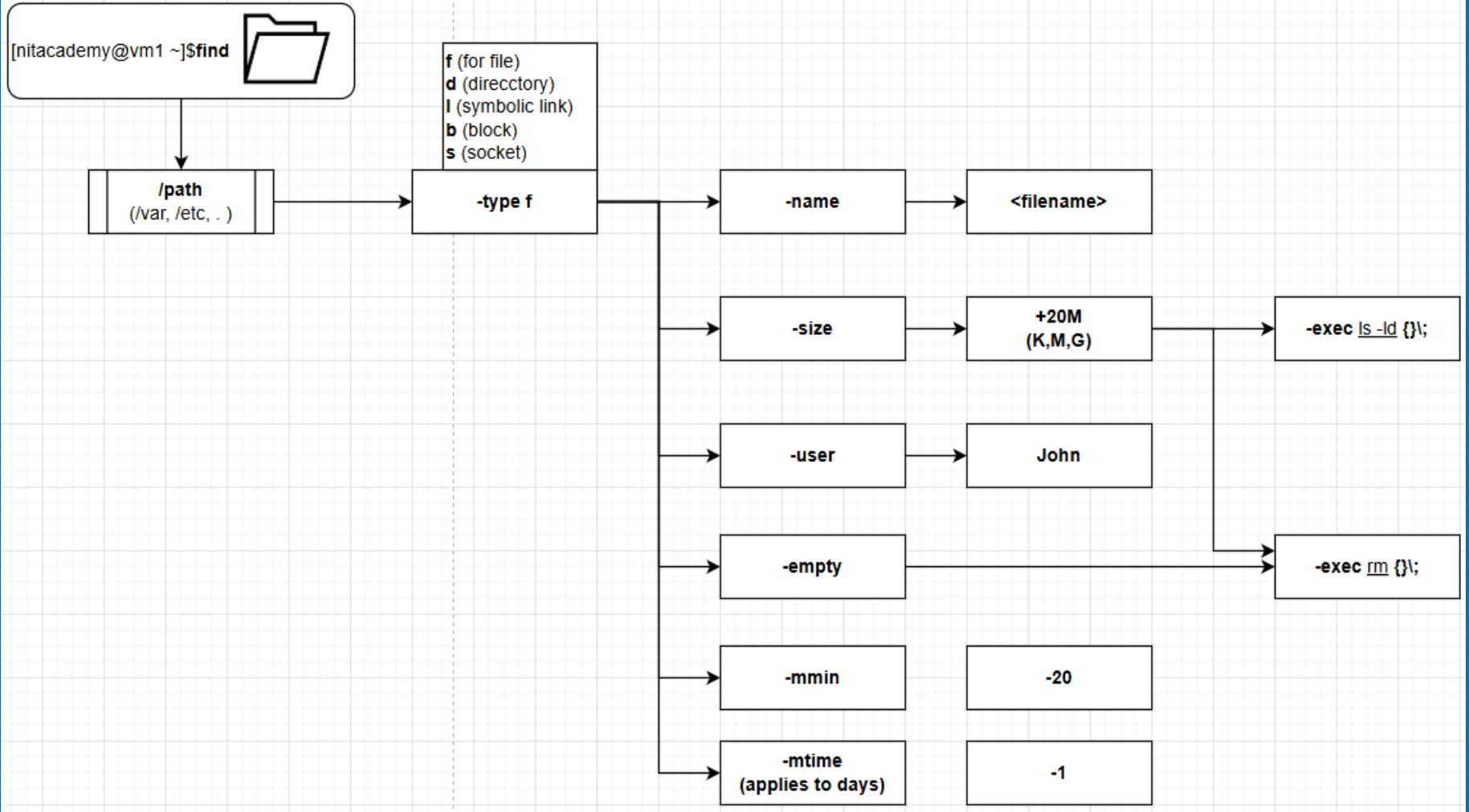
```
1 clear
2 cd /etc/
3 ls -l
4 man hostnamectl
5 hostnamectl set-hostname nitadmin
6 sudo hostnamectl set-hostname nitadmin
7 systemctl restart systemd-hostnamed
8 exit
9 hostname
10 hostname -i
```

recent

Running a command found in #history

```
436 ls -l /etc
437 ls -ld etc
438 ls -ld /etc
439 clear
440 stat /etc/passwd
441 du -s /etc/passwd
442 du -sh /etc/passwd
443 cat .bash_logout
444 vi /home/tom/.bash_logout
445 history
[nitadmin2@VM01 ~]$ !436
ls -l /etc
```

FIND COMMAND



PRACTICE FIND COMMAND

How to find all files and directories related to user john

```
[root@practicelab~]# useradd john
```

```
[root@practicelab~]# cd /home
```

```
[root@practicelab~]# find /home -type d -user john
```

How to find all the files that were modified in last 20 mins under “/var” /directory

```
[root@practicelab ~]# find /var -type f -mmin -20
```

How to find all the files that are older than 1 day under “/tmp”, and remove them

```
[root@practicelab ~]# find /tmp -type f -mtime -1 -exec rm -rf {} \;
```

PRACTICE FIND COMMAND

Find all the files that are larger than 5MB's in your whole system

```
[root@Practicelab~]# find / -type f -size +5M
```

File out all the files that are larger than 20MB's and their owners, so we can inform the users to manage them accordingly

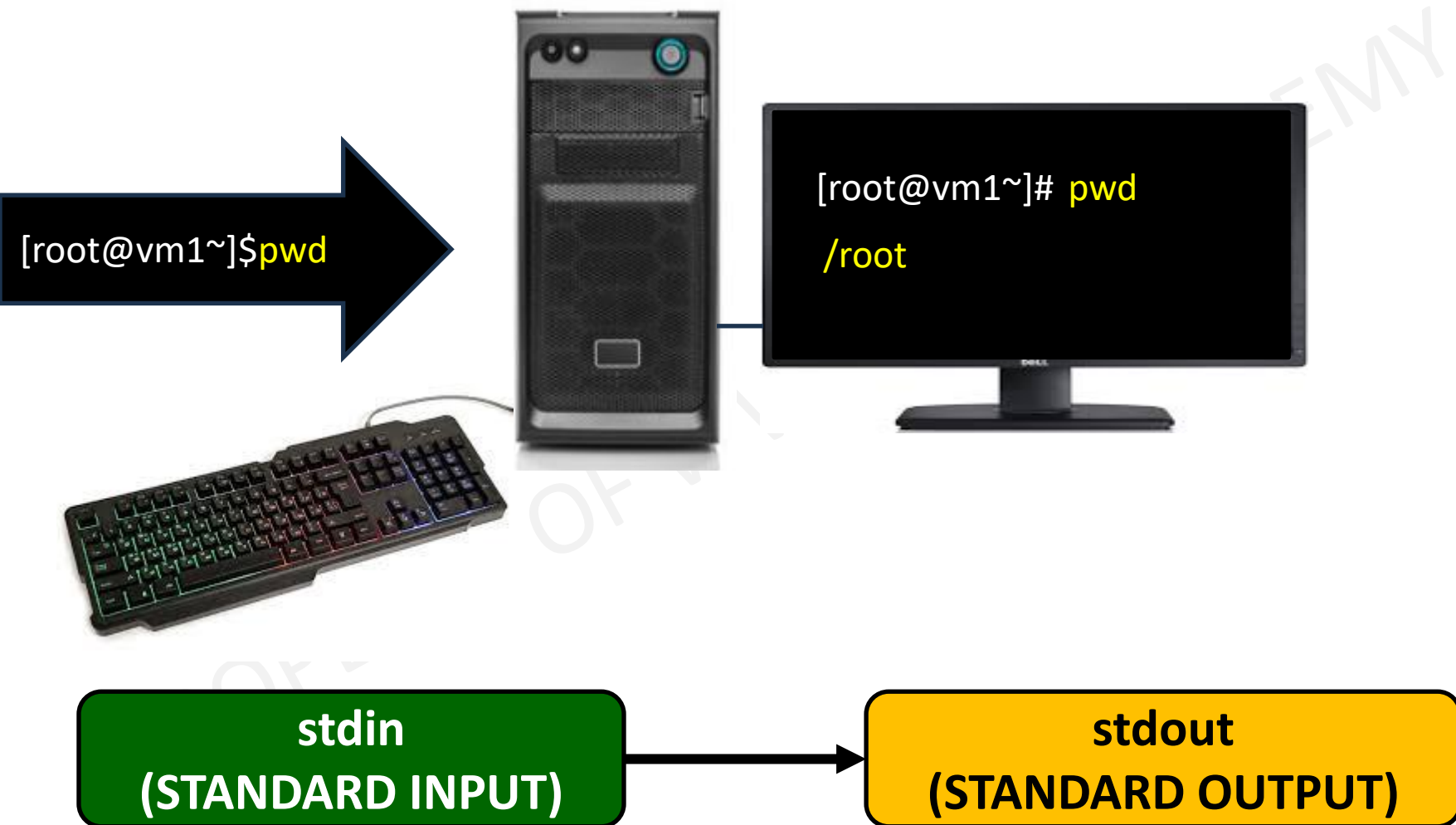
```
[root@Practicelab ~]# find / -type f -size +20M -exec ls -ld {} \;
```

STAT COMMAND

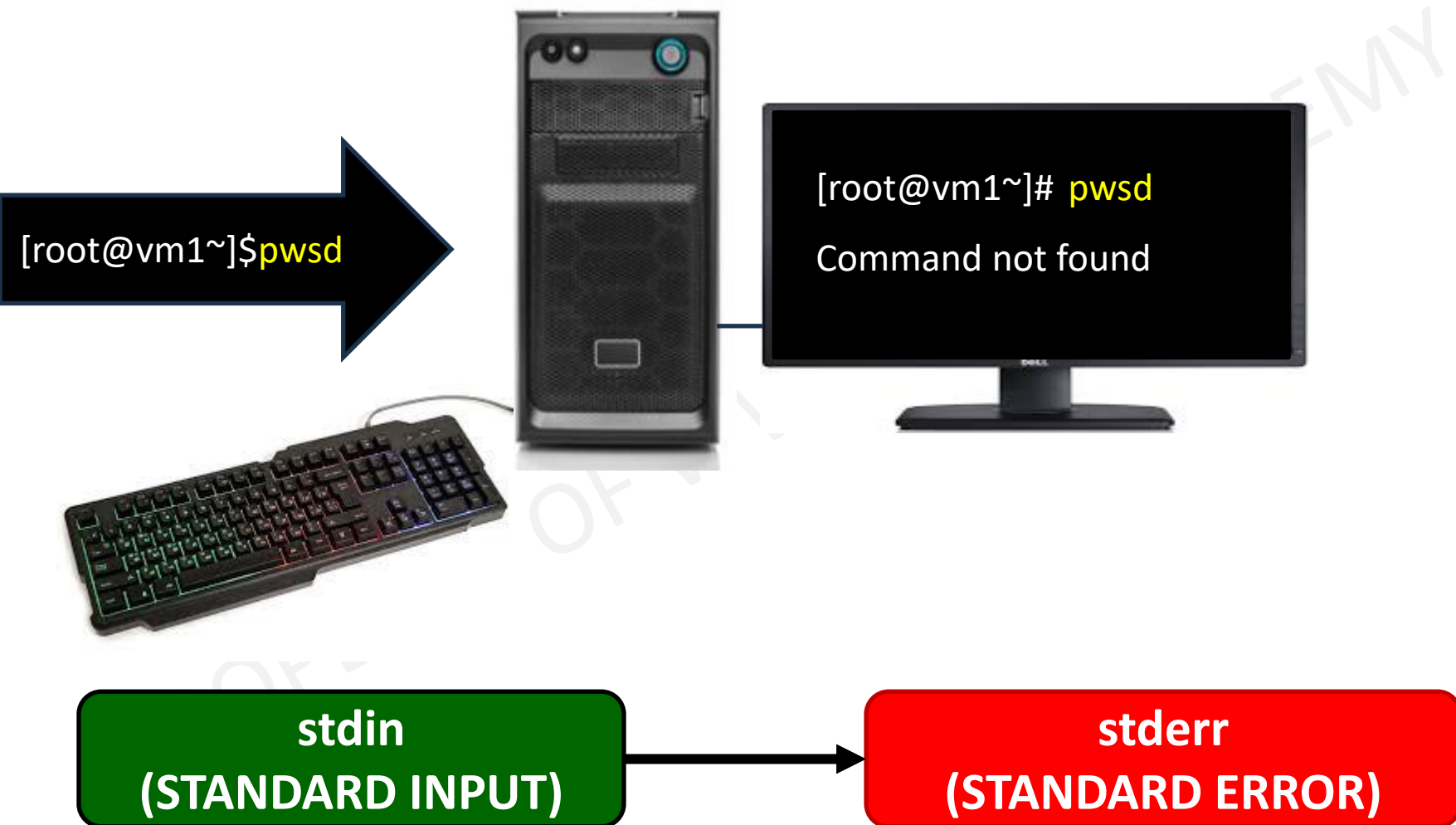
```
[root@practicelab ~]# ls -l
-rw-----. 1 root root 818 Apr 18 02:48 anaconda-ks.cfg
drwxr-xr-x. 3 root root 24 Jun 14 11:52 logs
-rw-r--r--. 1 root root 0 Jun 14 12:13 main.py
-rw-r--r--. 1 root root 26 Jun 14 12:17 output.txt
drwxr-xr-x. 2 root root 24 Jun 18 23:36 project

[root@practicelab ~]# stat main.py
  File: main.py
  Size: 0          Blocks: 0          IO Block: 4096   regular empty file
Device: fd00h/64768d  Inode: 16909384  Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 0/  root)  Gid: ( 0/  root)
Context: unconfined_u:object_r:admin_home_t:s0
Access: 2026-06-18 08:47:48.993631716 -0500
Modify: 2026-06-14 12:13:30.059713330 -0500
Change: 2026-06-14 12:13:30.059713330 -0500
 Birth: 2026-06-14 12:13:30.059713330 -0500
```

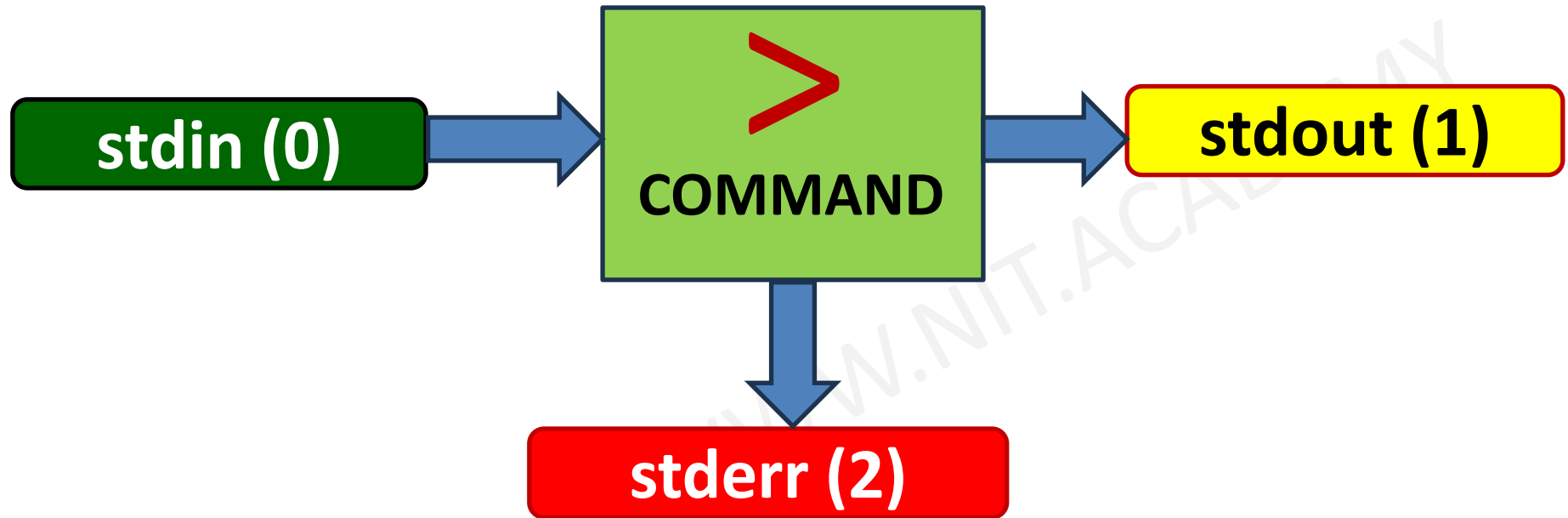
COMMANDS



COMMANDS



LINUX COMMANDS – HOW THEY WORK



File Descriptors: Every process opens three standard streams:

0 = Standard Input (stdin)

1 = Standard Output (stdout)

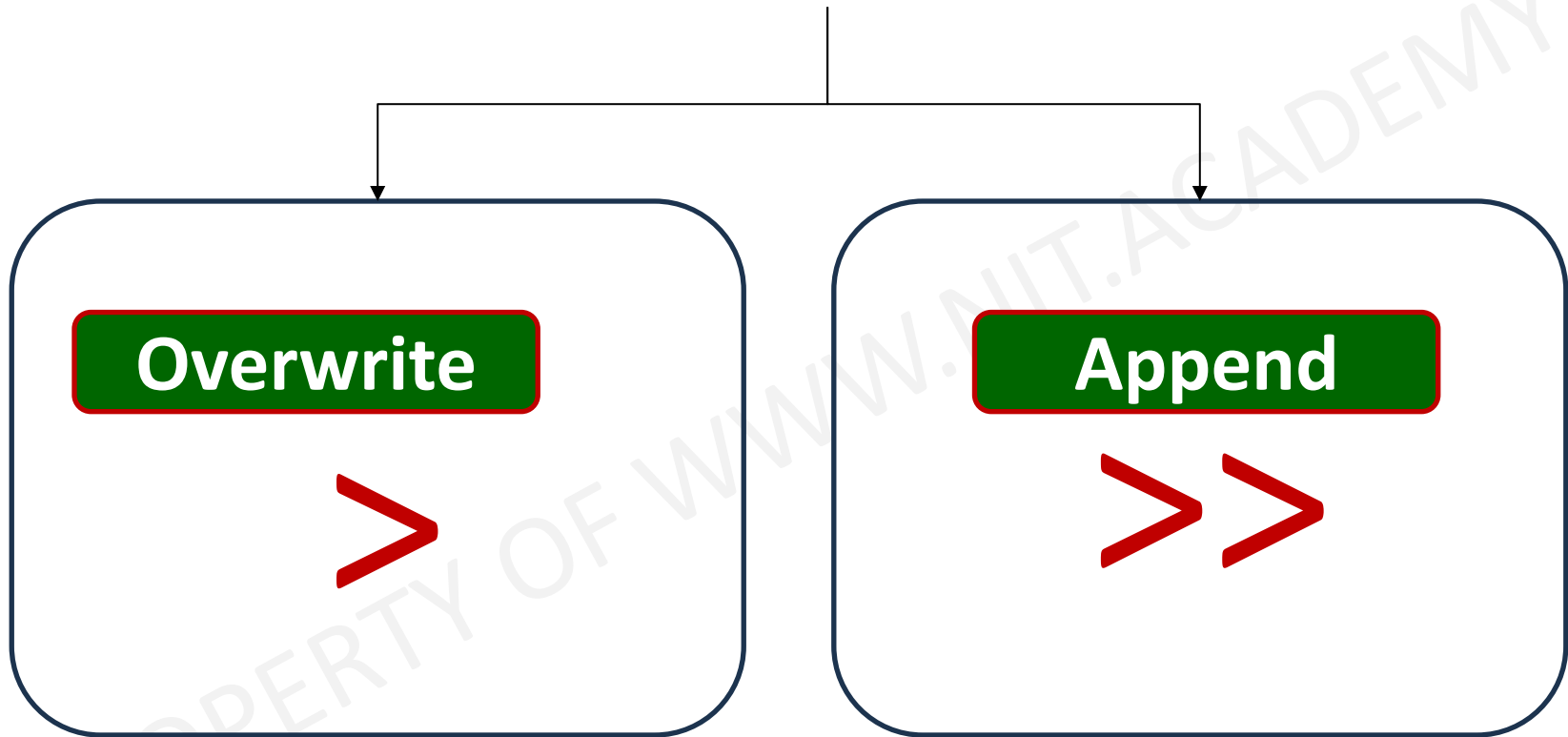
2 = Standard Error (stderr)

SUMMARY REFERENCE TABLE

Operator	What it does	Example
>	Redirects standard output (overwrites file)	ls > files.txt
>>	Redirects standard output (appends to file)	echo "done" >> log.txt
<	Takes input from a file	mysql db_name < backup.sql
2>	Redirects only errors	grep "secret" * 2> errors.txt
2>&1	Merges standard error into standard output (Classical Way)	command > file 2>&1
&>	Merges standard both output and errors (Replaces the classical way of 2>&1)	command &> output.txt
<<	Feeds multi-line text blocks (Here Doc)	cat << EOF
<<<	Feeds a single string (Here String)	bc <<< "5 + 5"

I/O REDIRECTION

Overwrite and Append are the two most popularly used I/O redirections. However, we will examine other redirects as they are also used by Linux Admins



PRACTICE REDIRECT - OVERWRITE

Let us create a Practice Directory:

```
[root@Practicelab~]# mkdir -p practicelab
```

```
[root@Practicelab~]# cd practicelab
```

Create practice file using touch command

```
[root@Practicelab ~]# touch output.txt
```

```
[root@Practicelab ~]# cat output.txt
```

Add information to "output.txt" file

```
[root@Practicelab ~]# System Logs > output.txt
```

```
-bash: System: command not found
```

```
[root@Practicelab ~]# echo "System Logs" > output.txt
```

```
[root@Practicelab ~]# cat output.txt
```



PRACTICE REDIRECT - OVERWRITE

Add information to "output.txt" file?

```
[root@Practicelab ~]# ls -l > output.txt
```

```
[root@Practicelab ~]# cat output.txt
```

#Let us try to add more information?

```
[root@Practicelab ~]# pwd > output.txt
```

```
[root@Practicelab ~]# cat output.txt
```

#Let us try to add more information?

```
[root@Practicelab ~]# date > output.txt
```

```
[root@Practicelab ~]# cat output.txt
```



We are redirecting BUT
Over-writing old
information and replacing
with new information

PRACTICE REDIRECT - APPEND

```
# Add information to "ouput.txt" file  
[root@Practicelab ~]# ls -l >> output.txt  
[root@Practicelab ~]# cat ouput.txt
```

```
#Let us try to add more information?  
[root@Practicelab ~]# pwd >> output.txt  
[root@Practicelab ~]# cat ouput.txt
```

```
#Let us try to add more information?  
[root@Practicelab ~]# echo "System Logs" >> output.txt  
[root@Practicelab ~]# cat ouput.txt
```



We are redirecting and appending new information to old information. NOT deleting old information.

STDOUT VS STDERR – MERGING CLASSICAL WAY

```
# Let us do something PRACTICAL! Change to Normal User
# Quiz what is the difference between "su – user vs su user"?
[root@Practicelab ~]# su – tom
Run:
[tom@Practicelab ~]$ tail -n 2 /etc/passwd /etc/shadow
==> /etc/passwd <==
tom:x:1003:1006::/home/tom:/bin/bash
labuser:x:1004:1006::/home/labuser:/bin/bash
tail: cannot open '/etc/shadow' for reading: Permission denied
#Let us try to add more information?
```

} is readable by all users (stdout) -> 1
} Requires root privileges (stderr) -> 2

```
# Let us Redirect to Separate Files
[root@Practicelab ~]# tail -n 2 /etc/passwd /etc/shadow > overwrite.txt 2> error.txt
[root@Practicelab ~]# cat overwrite.txt
[root@Practicelab ~]# cat error.txt

# Combining Error and output – Merging the streams
[root@Practicelab ~]# tail -n 2 /etc/passwd /etc/shadow > output.txt 2>&1
```


MERGING – MODERN WAY

Another easy way to Merge is below

Combining Error and output – Merging the streams

```
[root@Practicelab ~]# tail -n 2 /etc/passwd /etc/shadow &>1 output.txt
```

NOTE: Please don't forget where you are using ">" you can replace with ">>" as per your project requirements or work

Combining Error and output – Merging the streams

```
[root@Practicelab ~]# tail -n 2 /etc/passwd /etc/shadow &>>1 output.txt
```

HERE DOCUMENT '<<'

"<<" operator called "Here Document" allows you to feed multiple lines of text

```
[root@practicelab practicelab]# cat << EOF > css.txt
```

```
> #!/bin/bash
```

```
> echo "This is an executable file"
```

```
> echo "This file was created on $date"
```

```
> echo "How long has this server been running = $uptime"
```

```
> echo "What is the hostname: $hostname"
```

```
> EOF
```

```
[root@practicelab ~]# cat css.txt
```



SENDING DATA TO ABYSS

Linux has “/dev/null” that is considered the “Abyss of Linux”. If you absolutely don’t need to save the output or the errors then you can redirect to /dev/null

#Silence both output and errors completely

```
[root@practicelab practicelab]# ping 8.8.8.8 &>/dev/null
```

```
> #!/bin/bash
```

```
> echo "This is an executable file"
```

```
> echo "This file was created on $date"
```

```
> echo "How long has this server been running = $uptime"
```

```
> echo "What is the hostname: $hostname"
```

```
> EOF
```

```
[root@practicelab ~]# cat css.txt
```

DELETING (REMOVE) IN LINUX

#rm - file
#rm -rf - directory

Remember:

```
[root@Practicelab ~]# rmdir practicelab
```

```
rmdir: failed to remove 'practicelab': Directory not empty
```

```
[vm2@System-Admin ~]$ ls
a.txt  b.txt  dir1  file10  my_backup
[vm2@System-Admin ~]$ rm file10
[vm2@System-Admin ~]$ rm my_backup
rm: cannot remove 'my_backup': Is a directory
[vm2@System-Admin ~]$ rm -rf my_backup
[vm2@System-Admin ~]$ ls
a.txt  b.txt  dir1
```

SELINUX - PRACTICE LAB

How do you check details about SELinux identity?

[root@Practicelab ~]# **dnf install -y policycoreutils-python-utils**

Installed:

checkpolicy-3.6-1.el9.x86_64	policycoreutils-python-utils-3.6-5.el9.noarch
python3-audit-3.1.5-8.el9.x86_64	python3-distro-1.5.0-7.el9.noarch
python3-libsemanage-3.6-5.el9_6.x86_64	python3-policycoreutils-3.6-5.el9.noarch
python3-setools-4.4.4-1.el9.x86_64	python3-setuptools-53.0.0-15.el9.noarch

Complete!

semanage – SELinux Policy Management Tool

[root@Practicelab ~]# **semanage login -l**

Login Name	SELinux User	MLS/MCS Range	Service
__default__	unconfined_u	s0-s0:c0.c1023	*
root	unconfined_u	s0-s0:c0.c1023	*

Linux Security

Discretionary Access Control
(DAC)

Mandatory Access Control
(MAC)

Role Based Access Control
(RBAC)

DAC vs. MAC in Linux

Discretionary Access Control (DAC)

Owner controls access



In DAC, the **user** holds the keys—Flexible but risky.

Mandatory Access Control (MAC)

System enforces access



In MAC, the **System** holds the rules ♥ Secure but strict.



CONTROL ACCESS TO FILES

USER & FILE PERMISSIONS

Linux File Permissions

What are ACLs?

Linux File Permissions

-rwxrwx-x

Owner Group Other

File Permissions
Owner
Extended Attributes
SetUID/SetGID
Sticky Bit

ACLs



How do I change ownership?

-chmod-

File Ownership
chown, chgrp

..hcts-
chown, chgrp

Default Mask
-umask-

File Metadata
getattr, setattr

File Metadata
getattr, setattr

Discretionary Access Control (DAC)

- In Linux, **Discretionary Access Control (DAC)** refers to the security model in which **the owner** of a resource (file, directory, etc.) **has control over its access permissions**.
- DAC covers **file permissions, user/group permissions**,
- DAC also covers **Access Control Lists (ACLs)** — but there are a few *additional mechanisms* under DAC worth noting.

No.	Component	Purpose	Command Examples
1.0	umask	Control Default permissions	umask, /etc/profile
2.0	File permissions	Control read/write/execute access	chmod, ls -l
3.0	File ownership	Change file owner and group user/group	chown, chgrp
4.0	ACLs	Give special access specific user/group	setfacl, getfacl
5.0	Special bits	Use SetUID, SetGID and Sticky Bit	chmod 4755, chmod +t
6.0	Extended attributes	Add Metadata to files	getfattr, setfattr

```
# ls -l file
-rw-r--r-- 1 john unix 120 Nov 19 23:49 file
```

Diagram illustrating the components of the `ls -l` command output:

- File type:** `-` (regular file), `d` (directory)
- Owner (rw-):** `r` (read), `w` (write), `x` (execute)
- Owner:** `john`
- Owner's primary group:** `unix`
- Size:** `120`
- Date:** `Nov 19 23:49`
- File Name:** `file`

When we use the "touch" command to create a file and then run "ls -l" to view information about a file we see the following:

```
[root@practicelab practicelab]# touch file.txt
[root@practicelab practicelab]# ls -l file.txt
-rw-r--r--. 1 root root 0 Jun 14 17:41 file.txt
```

File Permissions make up about 30% of **RCHSA exam**! Easy to master if you follow this three step plan:



1. File type: these are basically only two (2)

- **Directory** -> if you see 'd' in the beginning it is a 'directory'
- **Files** -> if you see '-' in the beginning it is a 'file'

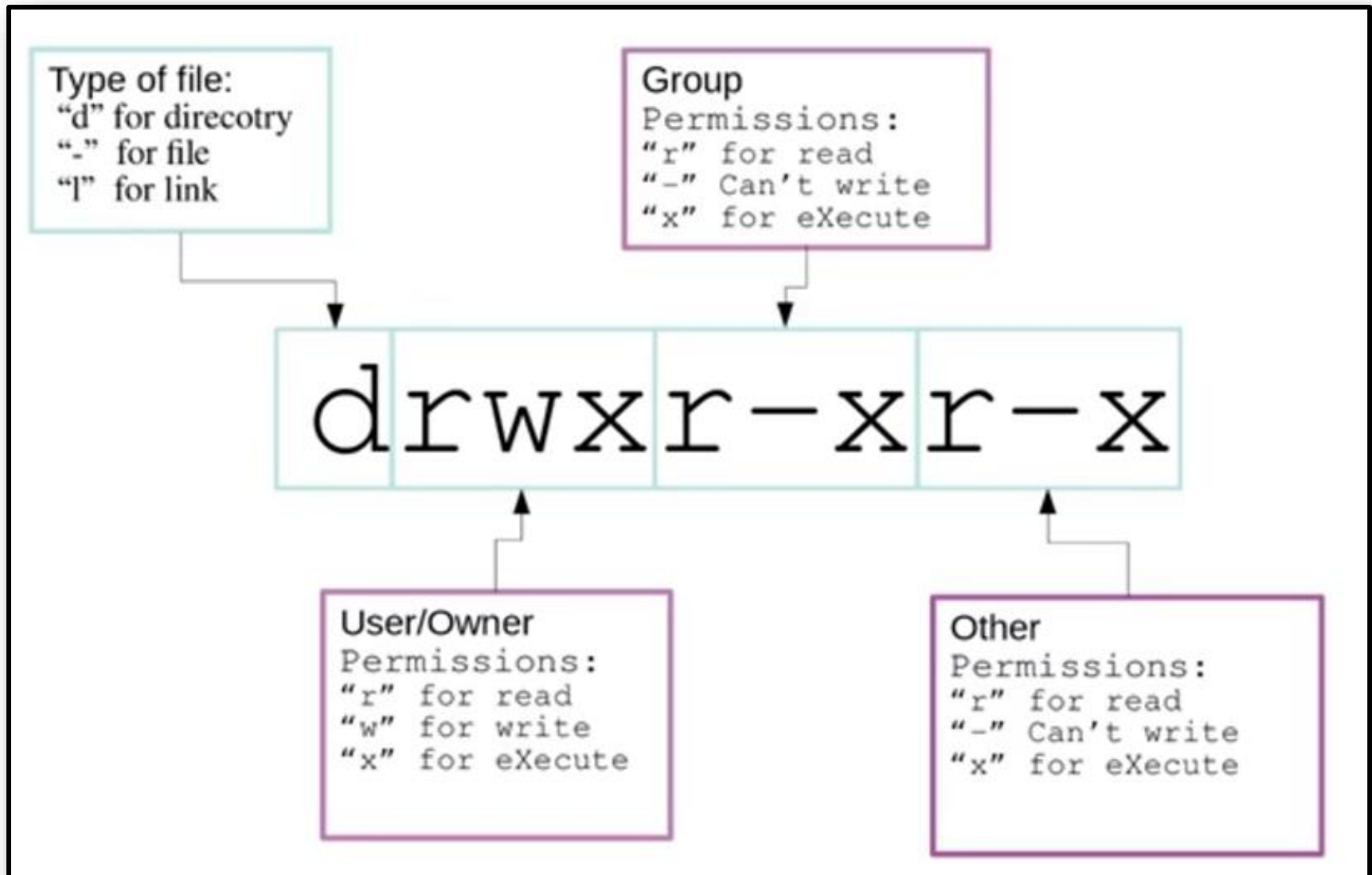
2. Permissions: these are three (3)

- **Read** -> represented with a 'r'
- **Write** -> represented with a 'w'
- **Execute** -> represented with a 'x'

3. These read/write/execute permissions are applied to three categories

- **owner** — **rwX** permissions **apply** only to the **user** who owns the file or directory.
- **group** — **rwX** permissions **apply** only to the **group** that has been assigned to the file or directory.
- **others** — **rwX** permissions **apply** only to all **other users** (everyone/public) on the system.

NORMAL PERMISSIONS



FILE PERMISSIONS (UMASK)



Linux has pre-defined permissions that can be modified using UMASK

<https://chmod-calculator.com/>

	FILE			DIRECTORY		
	User "u"	Group "g"	Other "o"	User "u"	Group "g"	Other "o"
Hard coded in Linux Kernel	6	6	6	7	7	7
umask	0	2	2	0	2	2
Default	6	4	4	7	5	5
	rw-	r--	r--	rwX	r-x	r-x

Run:

```
[root@Practicelab~]# umask -S
u=rwx,g=rx,o=rx
```

Run:

```
[root@Practicelab~]# umask
0022
```

Umask value => /etc/login.defs

022

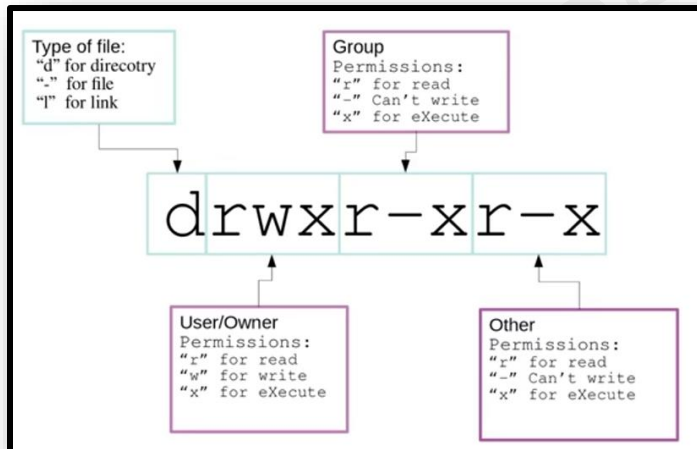
ABSOLUTE/SYMBOLIC MODE

Numeric / Absolute Mode

Permission	Octal Value
Read (r)	4
Write (w)	2
Execute (x)	1



```
[root@practicelab ~]# chmod 600 main.py
```



Symbolic Mode

Permission	Symbol	Operators
Read (r)	u,g,o	= , +, -
Write (w)	u,g,o	
Execute (x)	u,g,o	



```
[root@practicelab ~]# chmod u=rw,go= main.py
[root@practicelab ~]# chmod u=rw,go-rw main.py
```

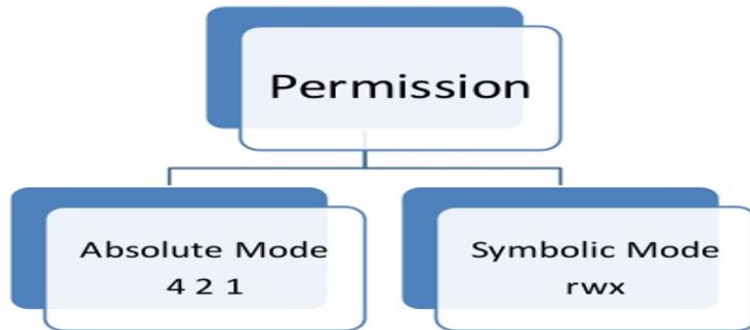
CHMOD COMMAND

Chmod – changing file permissions

The **chmod** command is used to change the permissions of a file or directory.

There are **2 ways to use the command** -

- Absolute mode (#chmod 4766 /etc/passwd)
- Symbolic mode (#chmod u=rwx, g=rwx, o=rwx)



CHMOD SYMBOLIC MODE

Operator	Description
+	Adds a permission to a file or directory
-	Removes the permission
=	Sets the permission and overrides the permissions set earlier.

The various owners are represented as -

User Denotations	
u	user/owner
g	group
o	other
a	all

SYMBOLIC/OCTAL COMPARISON

Compare

Command	Result	Numeric
<code>chmod u=rwx,g=rx,o=rx file</code>	<code>-rwxr-xr-x</code>	755
<code>chmod u=rws,g=rx,o=rx file</code>	<code>-rwsr-xr-x</code>	4755
<code>chmod u=rw,g=r,o=r file</code>	<code>-rw-r--r--</code>	644
<code>chmod u=rw,go=</code>	<code>-rw-----</code>	600

SYMBOLIC USE

Common Numeric ↔ Symbolic Equivalents

Numeric	Symbolic
777	<code>u=rwx,go=rwx</code>
755	<code>u=rwx,go=rx</code>
750	<code>u=rwx,g=rx,o=</code>
700	<code>u=rwx,go=</code>
644	<code>u=rw,go=r</code>
640	<code>u=rw,g=r,o=</code>
600	<code>u=rw,go=</code>

ABSOLUTE MODE - PRACTICE

Using "Numeric/Absolute Mode" change file permissions.

- Owner should have only read/write permission = $4+2 = 6$,
- Group should have no, read/write/execute permission = 0 ,
- Other users should have no, read/write/execute permission = 0

```
[root@practicelab ~]# ls -l main.py
```

```
--rw-r--r--. 1 root root  0 Jun 14 12:13 main.py
```

Let us change the permissions (600)

```
[root@practicelab ~]# chmod 600 main.py
```

```
[root@practicelab ~]# ls -l main.py
```

```
--rw-----. 1 root root  0 Jun 14 12:13 main.py
```

#Verify using Stat Command

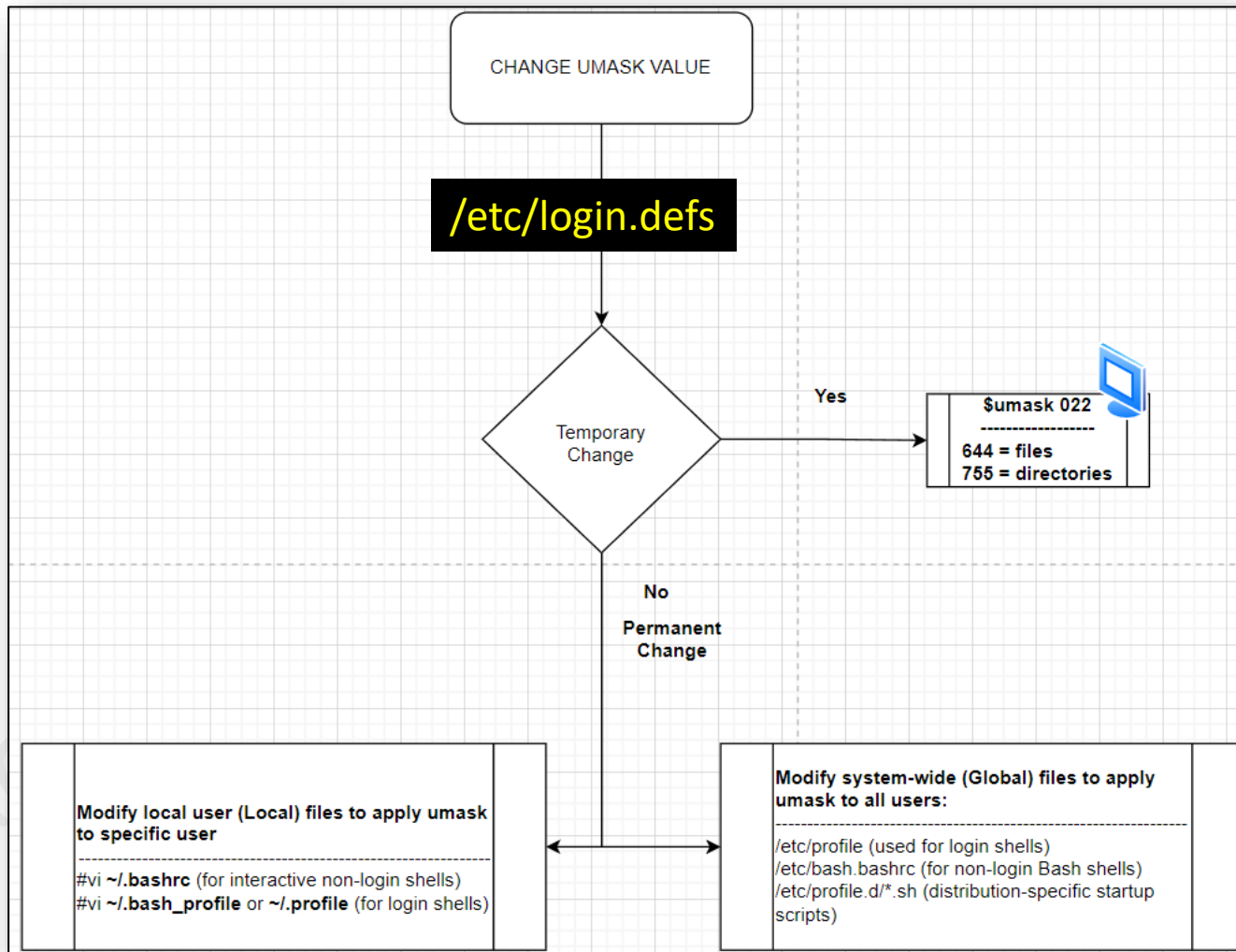
```
[root@practicelab ~]# stat main.py
```

SYMBOLIC WAY - PRACTICE

```
# Using "Symbolic mode" change file permissions.  
Owner should have only read/write permission, u=rw  
Group should have no, read/write/execute permission g=0  
Other users should have no, read/write/execute permission, o=0  
[root@practicelab ~]# ls -l projects  
--rw-r--r--. 1 root root  0 Jun 14 12:13 main.py  
  
# Let us change the permissions:  
[root@practicelab ~]# chmod u=rw,g=0,o=0 main.py  
[root@practicelab ~]# ls -l main.py  
--rw-----. 1 root root  0 Jun 14 12:13 main.py  
  
#Verify using Stat Command  
[root@practicelab ~]# stat main.py
```

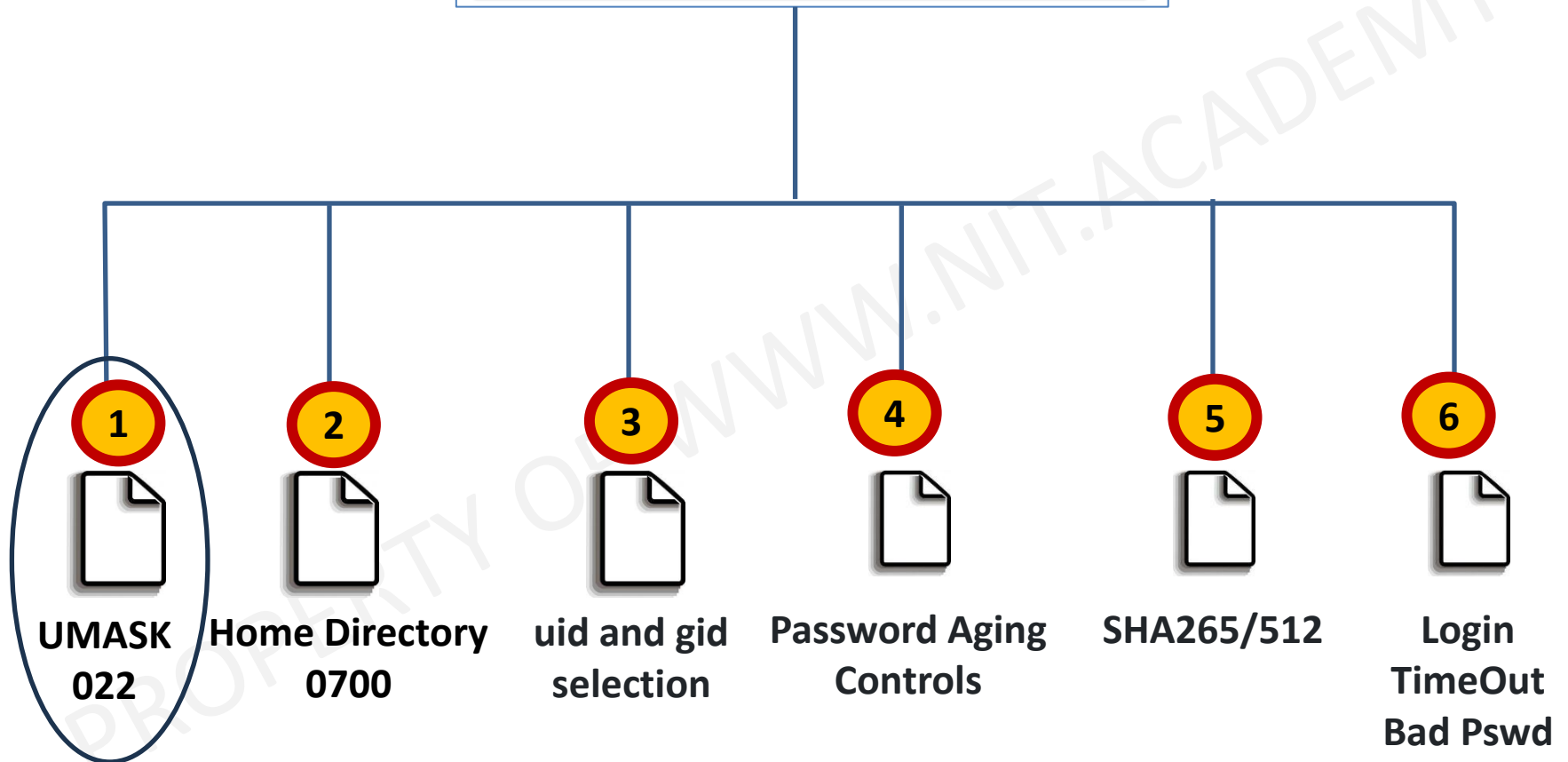
1.0 UMASK

The **umask** value, which determines the default permissions for newly created files and directories, can be changed in several places depending on the scope and persistence you want. Here's where you can change the umask value:



/ETC/LOGIN.DEFS - FILE

```
$cat /etc/login.defs
```



1.0 - PRACTICE UMASK

Check Current umask value:

```
[root@Practicelab~]# umask
```

```
0022
```

Create practice file

```
[root@Practicelab ~]# touch umask-file
```

```
[root@Practicelab ~]# ls -l umask-file
```

```
-rw-r--r--. 1 root root 0 Jun 12 23:45 umask-file
```

Create a practice directory

```
[root@Practicelab ~]# mkdir -p umask-dir
```

```
[root@Practicelab ~]# ls -ld umask-dir
```

```
drwxr-xr-x. 2 root root 6 Jun 12 23:46 umask-dir
```

PRACTICE UMASK

Change umask value temporarily

```
[root@Practicelab~]# umask -S
```

```
u=rwx,g=rx,o=rx
```

Create practice file

```
[root@Practicelab ~]# touch umask-file
```

```
[root@Practicelab ~]# ls -l umask-file
```

```
-rw-r--r--. 1 root root 0 Jun 12 23:45 umask-file
```

Create a practice directory

```
[root@Practicelab ~]# mkdir -p umask-dir
```

```
[root@Practicelab ~]# ls -ld umask-dir
```

```
drwxr-xr-x. 2 root root 6 Jun 12 23:46 umask-dir
```

CONFIGURATION - USERS

```
[root@Practicelab~]# cat /etc/login.defs
```

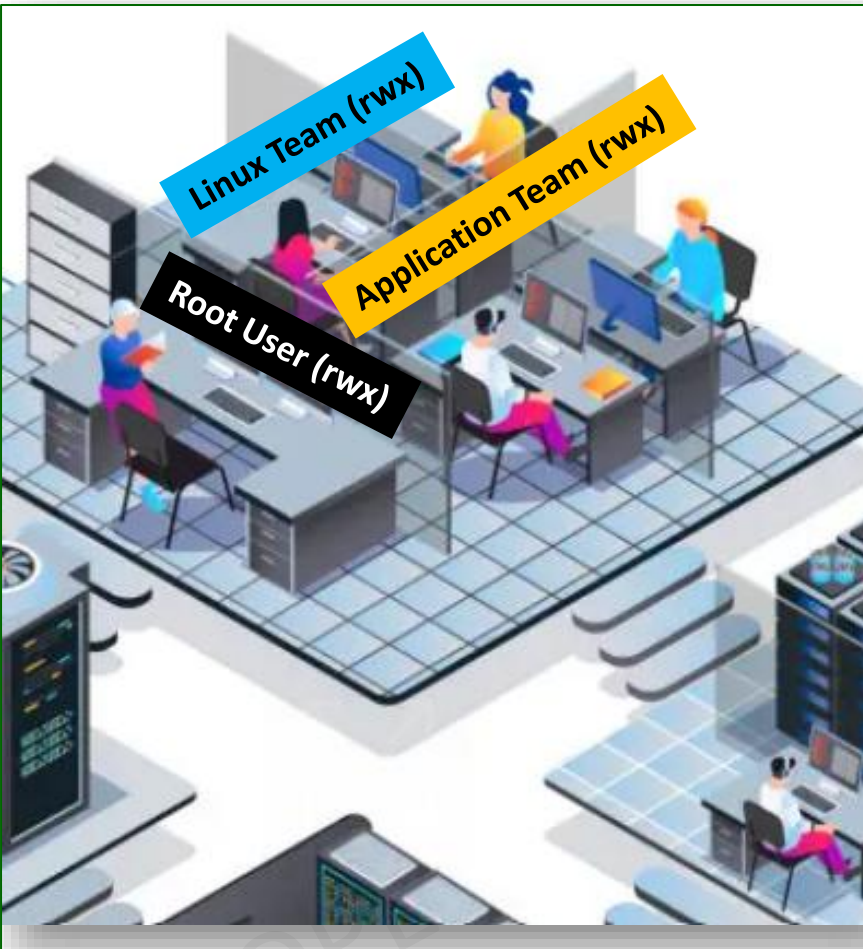
Please note that the parameters in this configuration file control the
behavior of the tools from the **shadow-utils** component. **None of these**
tools uses the PAM mechanism, and the utilities that use PAM (such as the
passwd command) should therefore be configured elsewhere. Refer to
/etc/pam.d/system-auth for more information.

This statement is referring to the behavior of certain system administration tools that are part of the **shadow-utils** package on Linux. The shadow-utils package provides utilities for managing user accounts, passwords, and authentication settings.

This means that **shadow-utils** utilities handle authentication and user management directly by modifying files like **/etc/passwd**, **/etc/shadow**, and **/etc/group**, **and does not use the PAM framework**.

Other utilities like **passwd use PAM** and must be configured separately in **/etc/pam.d/system-auth** or other PAM-related files.

2.0 LINUX POLICY – FILE PERMISSIONS



Carefully Read this!

When it comes to **file permissions** (rwx) Linux has a Policy that it applies on all files and directories. It enforces this Policy by implementing **umask** based on **/etc/login.defs** file.

Let us understand this clearly!

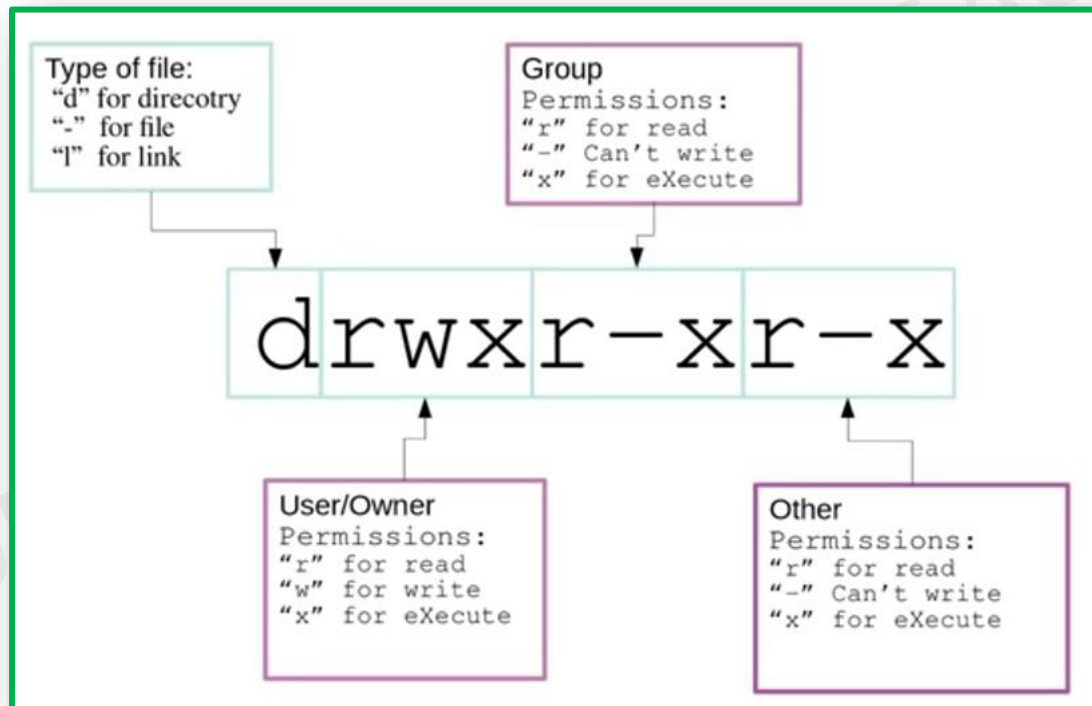
Linux hard-coded values (666/777) are run through => **/etc/login.defs** file. In this file the default value of umask = 0022

Linux hard-coded values run through the **/etc/login.defs** and subtracts the hard-coded value from the umask value giving the following permissions to files and directories:

- Permissions set on **Files**:
 $666 - 022 = \mathbf{644}$
- Permissions set on **Directories**:
 $777 - 022 = \mathbf{755}$

FILE PERMISSIONS

1. **Linux** on its own provides **Discretionary Access Control (DAC)**. This refers to the security model in which **the owner of a resource (file, directory, etc.) has control over its access permissions**
2. **DAC** comprises of six (6) types of controls
3. We will now examine how **Read, Write and Execute** Permissions can be applied by a **user to control files and directory access**



<https://chmod-calculator.com/>

WHAT COMMANDS ARE AFFECTED BY PERMISSIONS

There are **three (3)** permissions on any file and directory. Depending on the permission (rwx) the following Linux Line commands will execute:

1. **r** - means when a user runs these commands => "ls, cat, less, more, grep, tail, head", the user will be able to **read** a file.
2. **w** - means when a user runs these commands => "touch, vi, vim, cp, mv, >, >>, <<", the user will be able to **write** to that file
3. **x** - means when a user runs this command => "./<filename>", the user will be able to **execute** (run) a script.

FUNDAMENTAL CONCEPTS

Remember when it comes to **directories (folders)**, Linux splits “**ls**” and “**cat**” commands in a really interesting way!

Because **ls** and **cat** do two fundamentally different things, **they require different permissions to work on a folder**.

Here is the breakdown of how “Read” (r) and “Execute” (x) affect those two commands:

The **ls** command requires READ (r)

1. The **ls** command asks the system: “*Give me the list of filenames inside this folder.*” If you **remove read permission (-wx)**, **ls** will fail with Permission denied.
2. If you **have read permission**, **ls** works and will happily show you the names of the files inside.

The **cat** command requires EXECUTE (x)

1. The **cat** command doesn't care about the folder's list; it wants to bypass the folder and look *inside* a specific file (e.g., **cat folder/file.txt**). Passing through a folder to get to a file is called **traversal**, and that requires **Execute (r--)** permission.
2. If you **remove read permission** but **keep execute permission (--x)**, **ls** will fail, **but cat folder/file.txt will still work** (as long as you know the exact filename)!

PRACTICE LAB: UNDERSTANDING 644 AND 755

Objective:

Learn the difference between file permissions and directory permissions

Login into your VM via Guacamole and create two new users as root

```
[root@Practicelab~]# useradd dev1
```

```
[root@Practicelab~]# useradd dev2
```

Assign passwords as follows:

```
[root@Practicelab ~]# echo "redhat" | passwd --stdin dev1
```

```
[root@Practicelab ~]# echo "redhat" | passwd --stdin dev2
```

PRACTICE LAB: UNDERSTANDING 644 AND 755

Terminal 1 (User Dev1)

SSH into the machine:

```
>ssh user1@192.168.1.xxx
```

```
[dev1@Linux ~]$ whoami
```

```
dev1
```

Create a file

```
[dev1@Practicelab ~]$ echo "Linux Bootcamp" > notes.txt
```

```
[dev1@Practicelab ~]$ chmod 644 notes.txt
```

Create a directory

```
[dev1@Practicelab ~]$ mkdir project
```

```
[dev1@Practicelab ~]$ chmod 755 project
```

PRACTICE LAB: UNDERSTANDING 644 AND 755

Terminal 2 (User Dev1)

SSH into the machine:

```
>ssh user2@192.168.1.xxx
```

```
[dev2@Linux ~]$ whoami
```

```
dev2
```

Can dev2 read dev1's file?

```
[dev2@Linux ~]$ cat /home/dev1/notes.txt
```

```
cat: /home/dev1/notes.txt: Permission denied
```

Why is dev2 unable to view the files of dev1 when the permissions on dev1 are 644?

SO WHY THE “PERMISSION DENIED” ERROR?

- The issue isn't the **file** permissions; it's the **parent directory** permissions. In Linux, to read a file, a user must also have **execute (x) permissions** on every single directory in the path leading up to that file. Here is exactly what is happening and how to fix it.
- By default, on many Linux distributions, a user's home directory (like /home/dev1) is created with strict permissions (often 700 or 750). This prevents other users from even entering or looking inside that folder.
- Even though /home/dev1/notes.txt says "**Anyone can read me!**", user dev2 can't get past the front door of /home/dev1.
- You can check the permissions of the parent directory by running this command as **dev2 or root**:

```
[dev2@Linux ~]$ namei -l /home/dev1/notes.txt
f: /home/dev1/notes.txt
dr-xr-xr-x root root /
drwxr-xr-x root root home
drwx----- dev1 dev1 dev1 ← This is the blocker (no 'x' for others)
notes.txt - Permission denied
```

SO WHY THE “PERMISSION DENIED” ERROR?

How can we FIX this problem?

How to Fix It

To allow dev2 to read the file, dev1 needs to grant "execute" (traversal) permission to their home directory.

This is where you limit **WHO** can view your files. If you **ONLY** want **dev2** to have access without opening your home folder to the entire system, we will have to use an **Access Control List (ACL)**:

```
[dev1@Linux ~]$ setfacl -m u:dev2:x /home/dev1
```

Once dev1 applies either of those fixes, dev1's `cat /home/dev1/notes.txt` command will work perfectly for dev2

```
[dev2@Linux ~]$ cat /home/dev1/notes.txt
```

Linux Bootcamp

NOTE:

Because 644 = rw-r--**r--**

Dev2 is “**Others**” and the permission on “**Others**” is read-only!

PRACTICE LAB: UNDERSTANDING 644 AND 755

Task 2: Can dev2 modify dev1 files? What do you think? Modify means can dev2 “write” inside dev1’s file?

```
[dev2@Linux ~]$ echo "Testing" >> /home/dev1/notes.txt
```

```
-bash: /home/dev1/notes.txt: Permission denied ← Trying to write. Dev2 can NOT.
```

File Permissions are 644 = rw- r-r--

Others can read ONLY the contents of a file using (cat)

Others can NOT write – meaning create, delete or rename files

Others can NOT run any scripts for automation using this file. Meaning no execution permissions

PRACTICE LAB: UNDERSTANDING 644 AND 755

#So far we have tested file permissions. Let us see if dev2 had directory permissions on dev1 folders?

#Can dev2 enter dev1's directory? R

```
[dev2@Linux ~]$ cd /home/dev1/project
```

```
[dev2@Linux project]$ pwd
```

```
/home/dev1/project
```

Directory Permissions are 755 = rwx r-x r-x

- Others can read contents of this directory "ls" works

```
[dev2@Linux project]$ ls -ld /home/dev1/project
```

```
drwxr-xr-x. 2 dev1 dev1 6 Jun 18 22:36 /home/dev1/project
```

- Others can write – others can NOT create, delete or rename files in this directory
- Others can execute – meaning allows you to enter the directory (cd).

PRACTICE LAB – SETUP

Run the following as “root”

```
[root@Practicelab~]# useradd tester1
```

```
[root@Practicelab~]# useradd tester2
```

```
[root@Practicelab~]# useradd tester3
```

Add their passwords as follows:

```
[root@Practicelab ~]# echo “abc” | passwd –sdin tester1
```

```
[root@Practicelab ~]# echo “abc” | passwd –sdin tester2
```

```
[root@Practicelab ~]# echo “abc” | passwd –sdin tester3
```

PRACTICE LAB - SETUP

```
# Add a new group called linuxteam
[root@Practicelab ~]# groupadd linuxteam

# Assign linuxteam as a Secondary group to tester1 and tester2
[root@Practicelab ~]# usermod -G linuxteam tester1
[root@Practicelab ~]# usermod -G linuxteam tester2

# Create a Directory for "Testing" our work before implementing it real-time!
[root@Practicelab~]# mkdir -p /project
[root@Practicelab~]# cd /project
```

CHANGING FILE PERMISSIONS

As a 'root' user create a file. Remember we are in directory called "project"

```
[root@Linux project]# touch html.index
```

```
[root@Linux project]# ls -l html.index
```

```
-rw-r--r--. 1 root root 0 Jun 18 23:36 html.index
```

#Let us change the permissions from 644 to 600

```
[root@Practicelab ~]# chmod 600 html.txt
```

```
[root@Practicelab ~]# ls -l html.txt
```

```
-rw- --- ---
```

What does this mean? -rw- --- ---

Owner: read/write

Group : none

Others : none

TASK 2 – CHANGING DIRECTORY PERMISSIONS

```
# Now let us give execute permissions”  
[root@Practicelab ~]# chmod 755 html01.txt  
[root@Practicelab ~]# ls -l html01.txt  
-rwxr-xr-x
```

```
# What does this mean? -rwxr-xr-x  
Owner: read/write/execute  
Group : read/execute only  
Others : read/execute/ only
```

3.0 CHOWN COMMAND

Chown - Changing File Ownership

- You can change the owner of a file by using the **chown** command.
- [root@practicelab ~]# **chown user:group filename**
- [root@practicelab ~]# chown tom:devops /opt/process

Chown can also be used in place of chgrp command. Why is that? Because “chgrp” command only changes the group but “chown” can do more:

- Change **ownership** ONLY:
[root@practicelab ~]# **chown tester1: /opt/process**
- Change **group** ONLY
[root@practicelab ~]# **chown :linuxteam /opt/process**
- Change **ownership** and **group**:
[root@practicelab ~]# **chown tom:devops /opt/process**

CHOWN/CHGRP

Let us check ownership on a file

```
[root@Practicelab ~]# ls -l
```

Let us change owner to tester1

```
[root@Practicelab ~]# chown tester1 html01.txt
```

```
[root@Practicelab ~]# ls -l
```

#Let us change "tester1" group to linuxteam

```
[root@Practicelab ~]# chgrp linuxteam html01.txt
```

#Let us change "tester2" ownership AND group in single command:

```
[root@Practicelab ~]# chown tester2:linuxteam html01.txt
```


4.0 ACCESS CONTROL LIST (ACL)

1. **ACL (Access Control List)** in Linux allows root to assign **specific permissions to any:**
 - **User(s)**
 - **Group(s)**
2. ACLs work alongside traditional Linux file permissions, not replacing them. ACLs offer more specific control, but if there are conflicts, ACLs take precedence.
3. Not all filesystems support ACLs. To ensure ACL support, the filesystem must be mounted with the `acl` option:

Key Commands for Managing ACLs are Linux:

1. **getfacl**: This command is **like “ls”** used to display the ACLs of files or directories.

```
getfacl /path/to/file (or directory)
```

This will show the current ACL settings of the specified file or directory.

2. **setfacl**: This command is **like “chmod”** used to set or **modify ACLs on files**

```
setfacl -m u:username:rwX /path/to/file
```

This grants the user called username read, write, and execute permissions on `/path/to/file`

ACL CHEAT SHEET

3: Table of reference

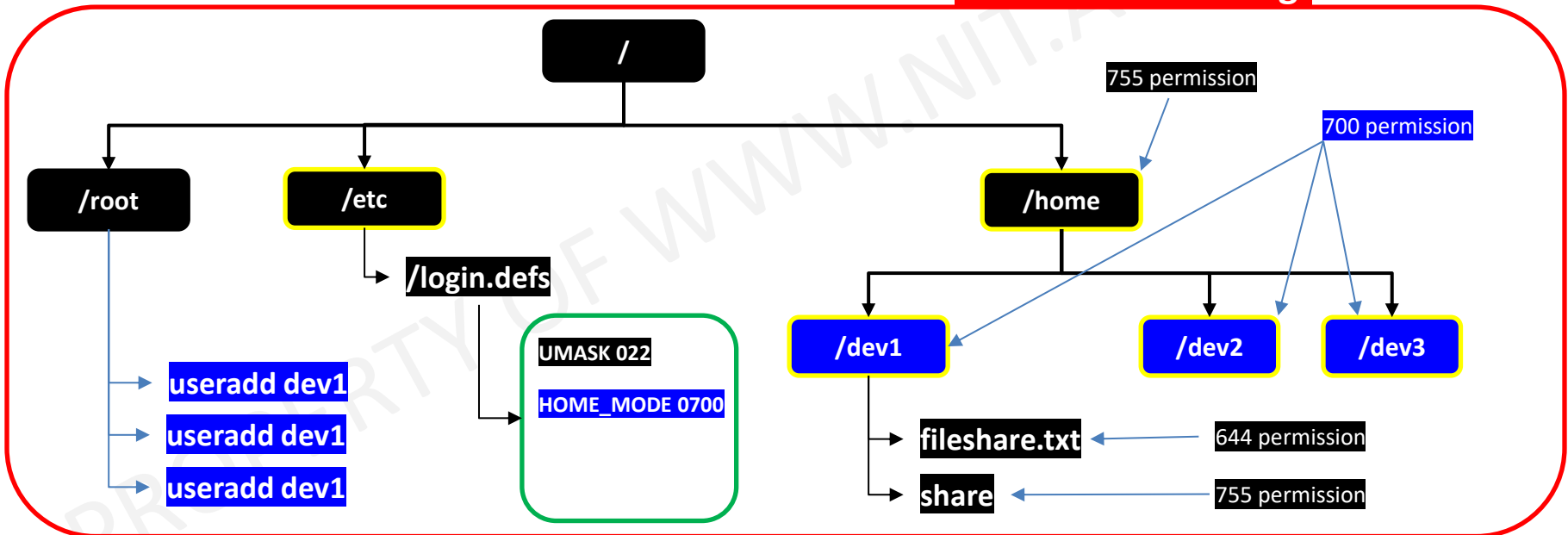
ACL Syntax Cheat Sheet		
-m	Add/Modify permissions AND Requires the permissions at the end.	setfacl -m u:dev2:rwX /home/dev1/project
-x	Remove specific user permissions AND Do <i>not</i> include the permissions	setfacl -x u:dev2 /home/The Clean Slatedev1/project
-s	Set/Replace ACLs - The Clean Slate	setfacl -s u:dev2:rw- /home/dev1/project/output.txt
-b	Remove ALL ACLs completely	setfacl -b /home/dev1/project
-d	Default – ACL's The Parent Blueprint - Any new file or subfolder created inside that directory from that moment forward will automatically inherit those permissions, no matter which user creates them.	setfacl -d -m u:dev2:rx /home/dev1/project

FUNDAMENTAL CONCEPT

Let us do that example from Beginning step-by-step:

1. We will create three users (dev1, dev2, dev3)
2. User dev1 will create a file (fileshare) and directory (share) in his/her home folder
3. User dev1 will then go into the directory share and create a file README.md
4. User dev1 will give ACL permission's ONLY to dev2
5. User dev2 will try to access dev1 files and directories before ACL is applied
6. User dev2 will try to access dev1 files and directories after ACL is applied
7. User dev3 will also check his/her access before and after.
8. Then dev1 will remove ALL ACL permissions

SELinux – Watchdog!



CHANGE YOUR MACHINE NAME

```
# SSH into your VM and let us change our machine name  
[root@Linux~]# hostnamectl set-hostname practicelab  
[root@Linux~]# exec bash  
[root@practicelab~]#
```

You have successfully changed your hostname and cleared a major question on the RHCSA Exam !!! **Welldone**

ACL PRACTICE LAB – SETUP

SSH into your VM and create three users as “root”

```
[root@Practicelab~]# useradd dev1
```

```
[root@Practicelab~]# useradd dev2
```

```
[root@Practicelab~]# useradd dev3
```

Add their passwords as follows:

```
[root@Practicelab ~]# echo “abc” | passwd –sdin dev1
```

```
[root@Practicelab ~]# echo “abc” | passwd –sdin dev2
```

```
[root@Practicelab ~]# echo “abc” | passwd –sdin dev3
```

OR simply use the passwd command

```
[root@Practicelab ~]# passwd dev1
```

```
[root@Practicelab ~]# passwd dev2
```

```
[root@Practicelab ~]# passwd dev3
```

TASK 1 – ACL (ACCESS CONTROL LIST)

```
# SSH from another window as User dev1. Create a file (fileshare) and directory (share)
[dev1@Practicelab ~]# touch fileshare
[dev1@Practicelab ~]# mkdir -p share
[dev1@Practicelab ~]$ ls -l
total 0
drwxr-xr-x. 2 dev1 dev1 24 Jun 19 01:42 share
-rw-r--r--. 1 dev1 dev1 0 Jun 19 01:41 fileshare.txt
```

What is this Dot => "." that we see at the end of the permissions?

Answer:

This is **Security Enhanced Linux (SELinux)** watching **"Watchdog"** these files and directories

Remember always: Permissions as per Umask are:

- 644 on files
- 755 on directories

TASK 2 – ACL (ACCESS CONTROL LIST)

#Let us check dev1 files/directories:

```
[dev1@practicelab ~]$ ll
```

```
-rw-r--r--. 1 dev1 dev1 11 Jun 19 11:39 fileshare.txt
```

```
drwxr-xr-x. 2 dev1 dev1 24 Jun 19 11:39 share
```

#User dev1 creates a file under `share` using “relative path”

```
[dev1@practicelab ~]$ echo "relative path" > share/output.txt
```

(Note! we are using Relative Path)

```
[dev1@practicelab ~]$ ls -l share/output.txt
```

```
-rw-r--r--. 1 dev1 dev1 5 Jun 19 11:41 share/output.txt
```

```
[dev1@practicelab ~]$ cat share/output.txt
```

relative path

#User dev1 creates a file under `share` using “absolute path”

```
[dev1@practicelab ~]$ echo "absolute path" > ~/dev1/share/file.txt
```

(Note! we are using Absolute Path)

```
bash: /home/dev1/dev1/share/file.txt: No such file or directory
```

(Why this error???) . ~ = /home/dev1

```
[dev1@practicelab ~]$ echo "absolute path" > ~/share/file.txt
```

```
[dev1@practicelab ~]$ cat share/file.txt
```

Absolute path

TASK 3 – ACL (ACCESS CONTROL LIST)

SSH as User dev2 in a separate Window and try to see dev1 file called **fileshare**

```
[dev2@practicelab ~]$ ls -l /home/dev1/fileshare
```

```
ls: cannot access '/home/dev1/fileshare': Permission denied
```

Let user dev2 try and access dev1 directory called **share**

```
[dev2@practicelab ~]$ ls -ld /home/dev1/share
```

```
ls: cannot access '/home/dev1/share': Permission denied
```

```
[dev2@practicelab ~]$ cat /home/dev1/fileshare.txt
```

```
cat: /home/dev1/fileshare.txt: Permission denied
```

```
[dev2@practicelab ~]$ cd /home/dev1/share
```

```
bash: cd: /home/dev1/share: Permission denied
```


TASK 4 – ACL (ACCESS CONTROL LIST)

#Let us run a Permissions Check to understand where we are getting our road block!

```
[dev2@practicelab ~]$ namei -l /home/dev1/fileshare
```

```
f: /home/dev1/fileshare
```

```
dr-xr-xr-x root root /
```

```
drwxr-xr-x root root home
```

```
drwx----- dev1 dev1 dev1 <- Here is the road block!
```

```
filesare - Permission denied
```

```
[dev2@practicelab ~]$ namei -l /home/dev1/share
```

```
f: /home/dev1/share
```

```
dr-xr-xr-x root root /
```

```
drwxr-xr-x root root home
```

```
drwx----- dev1 dev1 dev1 <- Here is the road block!
```

```
share - Permission denied
```

This is Because /home/dev1 has standard drwx----- permissions (700), dev2 is stopped right there. The system never even gets the chance to check what permissions are on the project folder or the filesare.txt file, because it can't even step inside /home/dev1.

TASK 5 – ACL (ACCESS CONTROL LIST)

Let us first check if ACL's are being applied to dev1 folder /home/dev1

```
[dev1@practicelab ~]$ getfacl /home/dev1
```

getfacl: Removing leading '/' from absolute path names

file: home/dev1

owner: dev1

group: dev1

user::rwx

group::---

mask::---

other::---

Let us allow ONLY dev2 using ACL

```
[dev1@practicelab ~]$ setfacl -m u:dev2:rwx /home/dev1
```

```
[dev1@practicelab ~]$ getfacl /home/dev1
```

getfacl: Removing leading '/' from absolute path names

file: home/dev1

owner: dev1

group: dev1

user::rwx

user:dev2:rwx

group::---

mask::rwx

other::---

TASK 6 – ACL (ACCESS CONTROL LIST)

Can dev2 get past /home/dev1 and access files and directories of dev1

```
[dev2@practicelab ~]$ ls -l /home/dev1/filesshare.txt
```

```
-rw-r--r--. 1 dev1 dev1 11 Jun 19 11:39 /home/dev1/filesshare.txt
```

```
[dev2@practicelab ~]$ cat /home/dev1/filesshare.txt
```

System log

```
[dev2@practicelab ~]$ ls -ld /home/dev1/share
```

```
drwxr-xr-x. 2 dev1 dev1 40 Jun 19 11:45 /home/dev1/share
```

```
[dev2@practicelab ~]$ cd /home/dev1/share
```

```
[dev2@practicelab share]$ ls -l
```

total 8

```
-rw-r--r--. 1 dev1 dev1 14 Jun 19 11:45 file.txt
```

```
-rw-r--r--. 1 dev1 dev1 14 Jun 19 11:42 output.txt
```

```
[dev2@practicelab share]$ cat file.txt
```

absolute path

```
[dev2@practicelab share]$ cat output.txt
```

relative path

TASK 7 – ACL (ACCESS CONTROL LIST)

Can dev3 get past /home/dev1 and access files and directories of dev1

```
[dev3@practicelab ~]$ ls -l /home
```

```
total 0
```

```
drwxrwx---+ 3 dev1  dev1  117 Jun 19 11:07 dev1
```

```
drwx----- 2 dev2  dev2   83 Jun 19 11:07 dev2
```

```
drwx----- 2 dev3  dev3   62 Jun 19 12:52 dev3
```

```
drwx----- 2 Ibrahim 1002 62 May 30 03:36 Ibrahim
```

```
drwx----- 3 student student 106 Jun 14 13:51 student
```

```
[dev3@practicelab ~]$ ls -l /home/dev1
```

```
ls: cannot open directory '/home/dev1': Permission denied
```

#All subsequent operations will fail

```
[dev3@practicelab ~]$ ls -l /home/dev1/fileshare.txt
```

```
ls: cannot access '/home/dev1/fileshare.txt': Permission denied
```

```
[dev3@practicelab ~]$ ls -ld /home/dev1/share
```

```
ls: cannot access '/home/dev1/share': Permission denied
```

```
[dev3@practicelab ~]$ cd /home/dev1/share
```

```
-bash: cd: /home/dev1/share: Permission denied
```

TASK 8 – ACL (ACCESS CONTROL LIST)

```
# Let us remove ONLY a specific user
[dev1@practicelab ~]$ getfacl /home/dev1
getfacl: Removing leading '/' from absolute path names
# file: home/dev1
# owner: dev1
# group: dev1
user::rwx
user:dev2:rwx
group::---
mask::rwx
other::---

[dev1@practicelab ~]$ setfacl -x u:dev2 /home/dev1
[dev1@practicelab ~]$ getfacl /home/dev1
getfacl: Removing leading '/' from absolute path names
# file: home/dev1
# owner: dev1
# group: dev1
user::rwx
group::---
mask::---
other::---
```

TASK 9 – ACL (ACCESS CONTROL LIST)

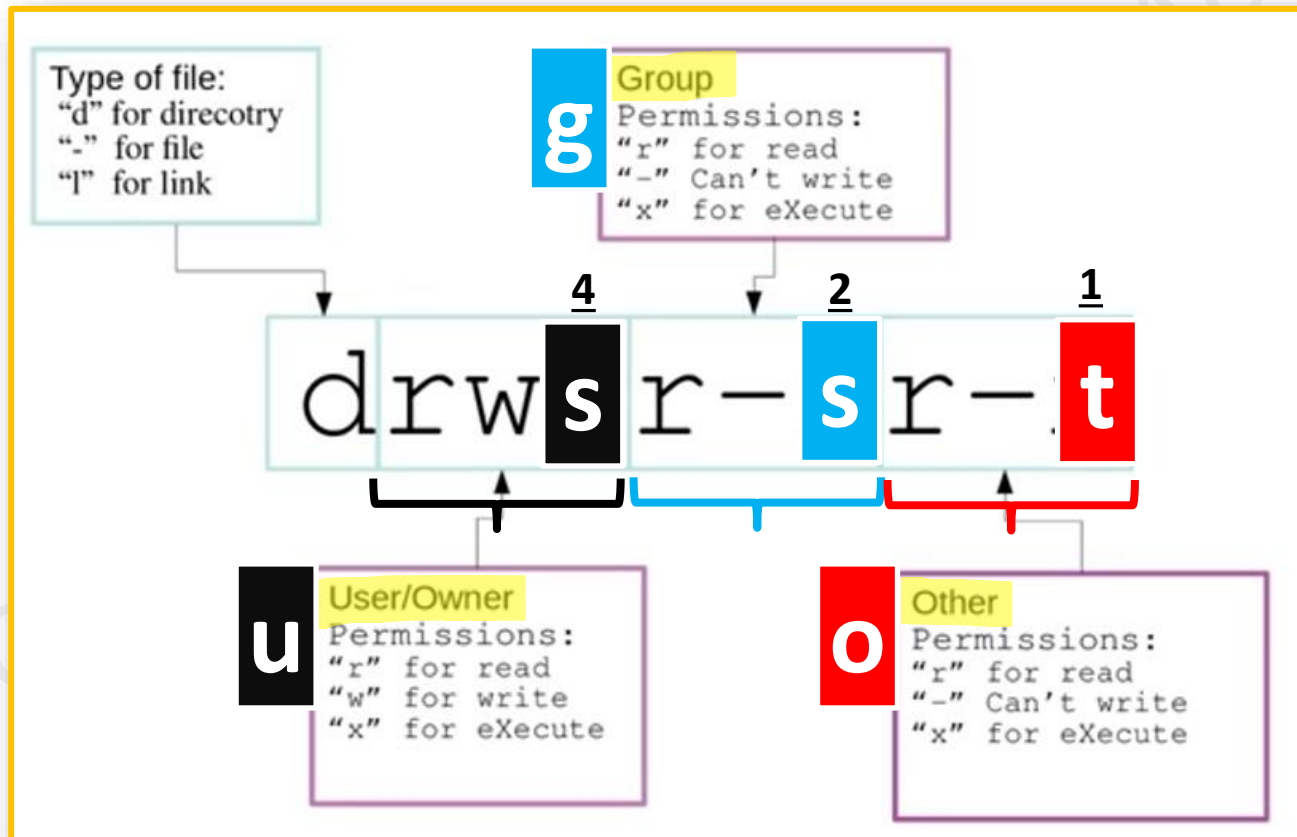
```
# Let us remove all ACL's applied:
[dev1@practicelab ~]$ setfacl -m u:dev2:rwX /home/dev1
[dev1@practicelab ~]$ getfacl /home/dev1
getfacl: Removing leading '/' from absolute path names
# file: home/dev1
# owner: dev1
# group: dev1
user::rwX
user:dev2:rwX
group::---
mask::rwX
other::---

[dev1@practicelab ~]$ setfacl -b /home/dev1
[dev1@practicelab ~]$ getfacl /home/dev1
getfacl: Removing leading '/' from absolute path names
# file: home/dev1
# owner: dev1
# group: dev1
user::rwX
group::---
other::---
```

5.0 SPECIAL PERMISSIONS

- Set SUID = 4 (Set special access permissions for Users)
- Set SGID = 2 (Set special access permissions for Groups)
- Sticky Bit = 1 (Set special access permission for Others)

NOTE: Run **STAT** command to get full details on a file or directory



SUID

fety™

- SUID works on FILES only it has NO effect on directories.
- When the setuid bit is used, the behavior of the file is modified. **For example**, if the file is owned by “root” and that file has SUID bit set, the file runs with root privileges. Meaning a user can execute the file with root privileges. This is a Security and Vulnerability issue.

```
[root@PracticeLab ~]# ls -l /usr/bin/passwd  
rwsr-xr-x. 1 root root 32656 May 14 2022 /usr/bin/passwd
```

NOTE: When an executable file with SUID is executed then the resulting process will have the permissions of the owner of the command, not the permissions of the user who executes the command

-S (capital S) => no executable permission applied to that file

- Check which commands have “s” bit set on them:

```
[root@PracticeLab ~]# find /usr/bin/ -perm -4000
```

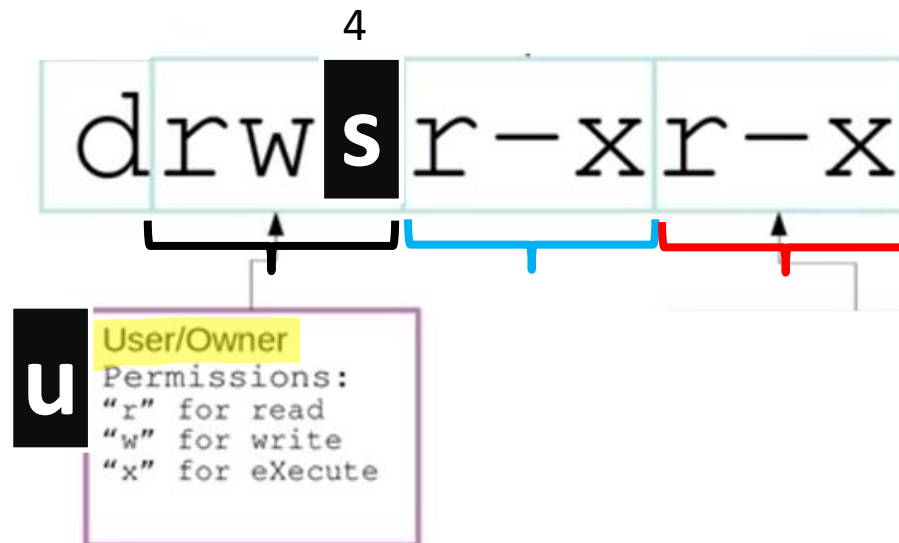
- Commands to set “s” or “S” bit
- Absolute mode: [root@PracticeLab ~]# **chmod 4xxx filename**
- Relative Mode : [root@PracticeLab ~]# **chmod u+s filename**

SPECIAL PERMISSIONS

- Set SUID = 4
- Set GUID = 2
- Sticky Bit = 1

NOTE: SETUID is not used on “ls” command because it is only reading. Whereas “**passwd, su, sudo, gpasswd**” command has a **security risk**.

- **Lowercase s (rws)**: Means the file has SUID set and the underlying file is executable (x). This is normal and functional.
- **Uppercase S (rwS)**: Means SUID is set, but the underlying execution bit (x) is missing. This is a configuration error—the SUID bit is useless because nobody has permission to execute the file anyway!



TASK 4 – SPECIAL PERMISSIONS

```
# As root user examine the passwd command:
[root@practicelab ~]# whereis passwd
passwd: /usr/bin/passwd /etc/passwd /usr/share/man/man1/passwd.1ossl.gz
/usr/share/man/man1/passwd.1.gz
[root@practicelab ~]# ls -l /usr/bin/passwd
-rwsr-xr-x. 1 root root 32656 May 14 2022 /usr/bin/passwd
```

NOTE: This “s” means that when a normal users runs passwd, it runs with root privilege so they can change their password.

```
[dev1@practicelab ~]$ passwd dev1
passwd: Only root can specify a user name.
[dev1@practicelab ~]$ passwd
Changing password for user dev1.
Current password:
New password: London1@ <- You can not see but for this lab we will use this password
BAD PASSWORD: The password fails the dictionary check - it is based on a dictionary word
New password:
```

FUNDAMENTAL CONCEPTS

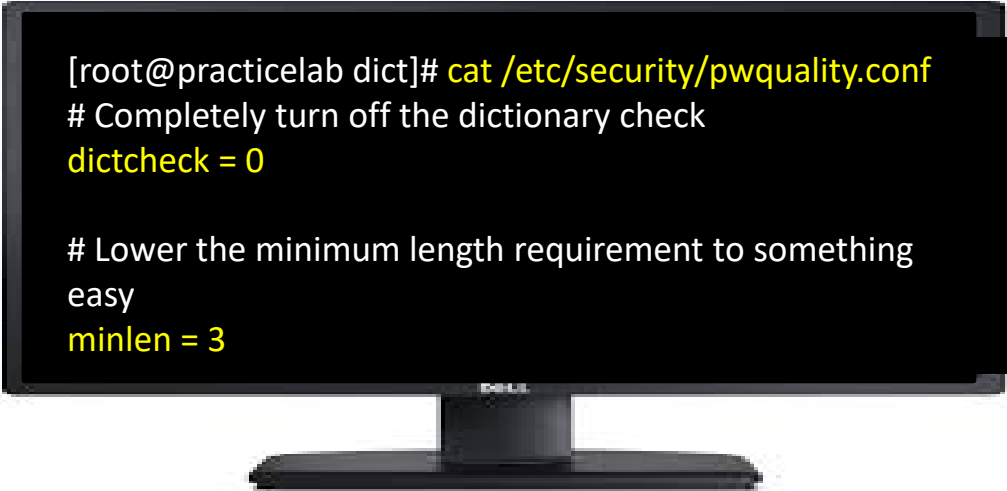
BAD Password is a Blocker!

This message is coming from **pam_pwquality** (or **pam_cracklib** on older systems), which is the **Pluggable Authentication Module (PAM)** responsible for **enforcing password strength rules** on enterprise Linux systems.

Even though it says BAD PASSWORD, it hasn't locked dev1 out or crashed. It's just rejecting dev1 choice. So, what next?

In modern Linux systems:

- **/etc/security/pwquality.conf** handles the password complexity (like **dictionary checks**),
- **/etc/login.defs** only handles **password aging/expiration**.



```
[root@practicelab dict]# cat /etc/security/pwquality.conf
# Completely turn off the dictionary check
dictcheck = 0

# Lower the minimum length requirement to something
easy
minlen = 3
```

FUNDAMENTAL CONCEPTS

The Division of Labor

`/etc/security/pwquality.conf` (The Complexity Enforcer)

- This file dictates **what the password must look like**. The PAM module checks this file *the exact moment* a user types a new password. It controls:
 - **dictcheck**: Checking against dictionary words (the rule you just hit).
 - **minlen**: Minimum password length.
 - **minclass**: How many character classes (lowercase, uppercase, digits, symbols) are required.

`/etc/login.defs` (The Aging Enforcer)

- This file dictates **how long a password is valid**. It has absolutely no idea if your password is "password123" or a highly secure 30-character string. It only controls time metrics for new users:
 - **PASS_MAX_DAYS**: How many days a user can keep a password before being forced to change it.
 - **PASS_MIN_DAYS**: How many days a user must wait before they are allowed to change it again.
 - **PASS_WARN_AGE**: How many days in advance the system starts warning the user that their password is expiring.

Why are there password settings in `/etc/login.defs` if they don't work?

- If you scroll through `/etc/login.defs`, you might see legacy variables like `PASS_MIN_LEN`.
- Here is the truth: **Those complexity settings inside `/etc/login.defs` are deprecated and completely ignored by modern systems**. Decades ago, the standard `passwd` command used to read `/etc/login.defs` directly to check password length. But when Linux switched to **PAM (Pluggable Authentication Modules)**, all complexity checking was handed over to modular plug-ins like `pam_pwquality`.
- PAM completely ignores the old limits in `login.defs`. If you try to change `PASS_MIN_LEN` in `login.defs` today, it will have zero effect on your students' password behavior.

FUNDAMENTAL CONCEPTS

The root user is a SUPER administrator, and the system will never block root from doing its job—even if it's a bad idea.

Here is exactly how the passwd program handles this exception under the hood.

The Execution Flow: Checking vs. Enforcing

When you run `passwd dev1` as root and type a weak password like **abc**, the system follows a two-step process:

Step 1: The Quality Check (PAM)

- The `passwd` command hands your input over to the PAM module (`pam_pwquality`). PAM looks up `abc` or `London1@`, references `/etc/security/pwquality.conf` and the system dictionary, and realizes it's completely insecure.
- PAM returns a status to the system saying: *"This is a terrible password!"* Because of that, the `passwd` program prints the BAD PASSWORD warning on your screen.

Step 2: The Root Privilege Check (The Exception)

- This is where the magic happens. Before the `passwd` program rejects the change, it checks the **UID (User ID)** of the person running the command.
- **If UID is NOT 0 (Standard User):** The program says, *"You are a regular user. I am blocking this change."* It exits and forces them to try again.
- **If UID IS 0 (Root User):** The program says, *"I've warned you that this is a bad password. But since you are root, I assume you have a specific architectural reason for doing this. Your wish is my command."* It overrides the PAM failure, writes the hash of `abc` directly into `/etc/shadow`, and saves it anyway.

Why does this matter?

This is a critical lesson in learning user administration:

- **Root is absolute:** Root can assign empty passwords, weak passwords, or duplicate UIDs. The OS expects the administrator to be the final authority.

DICTIONARY

- To save disk space, minimal server installations omit the optional text dictionaries package. However, the password checker **still works** because the optimized binary databases (/usr/share/cracklib/pw_dict.pwd) are packaged separately and are present.

Let us run these commands to check:

```
[root@practicelab dict]# ls -l /usr/share/cracklib/
```

```
total 9600
```

```
-rw-r--r--. 1 root root  360 Aug 18  2015 cracklib.magic
```

```
-rw-r--r--. 1 root root  1024 May 31  2022 cracklib-small.hwm
```

```
-rw-r--r--. 1 root root 260003 May 31  2022 cracklib-small.pwd
```

```
-rw-r--r--. 1 root root 13704 May 31  2022 cracklib-small.pwi
```

```
-rw-r--r--. 1 root root  1024 May 31  2022 pw_dict.hwm
```

```
-rw-r--r--. 1 root root 9047901 May 31  2022 pw_dict.pwd
```

```
-rw-r--r--. 1 root root 491360 May 31  2022 pw_dict.pwi
```

```
[root@practicelab dict]# cracklib-uncheck /usr/share/cracklib/pw_dict | less
```

```
-bash: cracklib-uncheck: command not found
```

```
[root@practicelab dict]# dnf install words -y
```

<- RHCSA EXAM QUESTION

SETUID

Now that “dnf install word -y” has been installed. Let us see how many words are in this dictionary:

```
[root@practicelab ~]# wc -l /usr/share/dict/words
```

```
479826 /usr/share/dict/words
```

#Let us scroll through the dictionary

```
[root@practicelab ~]# less /usr/share/dict/words
```

(press ‘q’ on your keyboard to always ‘quit’)

```
[root@practicelab ~]# head /usr/share/dict/words
```

```
[root@practicelab ~]# head -n 5 /usr/share/dict/words
```

Note: this will give you 10 records from the top

Note: this will give you “n” records from the top

```
[root@practicelab ~]# tail /usr/share/dict/words
```

```
[root@practicelab ~]# tail -n 5 /usr/share/dict/words
```

Note: this will give you 10 records from the bottom

Note: this will give you “n” records from the bottom

Check

```
[root@practicelab ~]# cracklib-check <<< "London1@"
```

```
London1@: it is based on a dictionary word
```

LESS/MORE COMMANDS

Directional Navigation (The Big One)

- **more is forward-only:** It allows you to move forward through a file line-by-line (using Enter) or screen-by-screen (using Spacebar). If you accidentally scroll past a line you wanted to read, **you cannot scroll backward**. You have to quit the program and restart it.
- **less is fully bidirectional:** It allows you to move freely in *any* direction. You can use your Up/Down arrow keys, Page Up/Page Down keys, or mouse wheel to scroll forward and backward seamlessly.

2. Speed and Memory Management

- **more loads the entire file:** If you try to open a massive 10 GB log file with more, the system tries to read and load the whole file into memory before showing you the first page. This can cause your terminal to freeze or lag severely.
- **less lazy-loads the file:** It only loads the exact chunk of text required to fill your current terminal window. Because it doesn't read the entire file upfront, **less opens a 10 GB file instantly**, making it vastly superior for production troubleshooting.

3. Searching Capabilities

- **more search is limited:** You can search forward for a term using /pattern, but jumping between multiple matches is clunky.
- **less search is powerful:** You can search forward with /pattern or backward with ?pattern. Once you find a match, you can tap n to jump to the next match, or N to jump to the previous match. It also highlights all matches on the screen automatically.

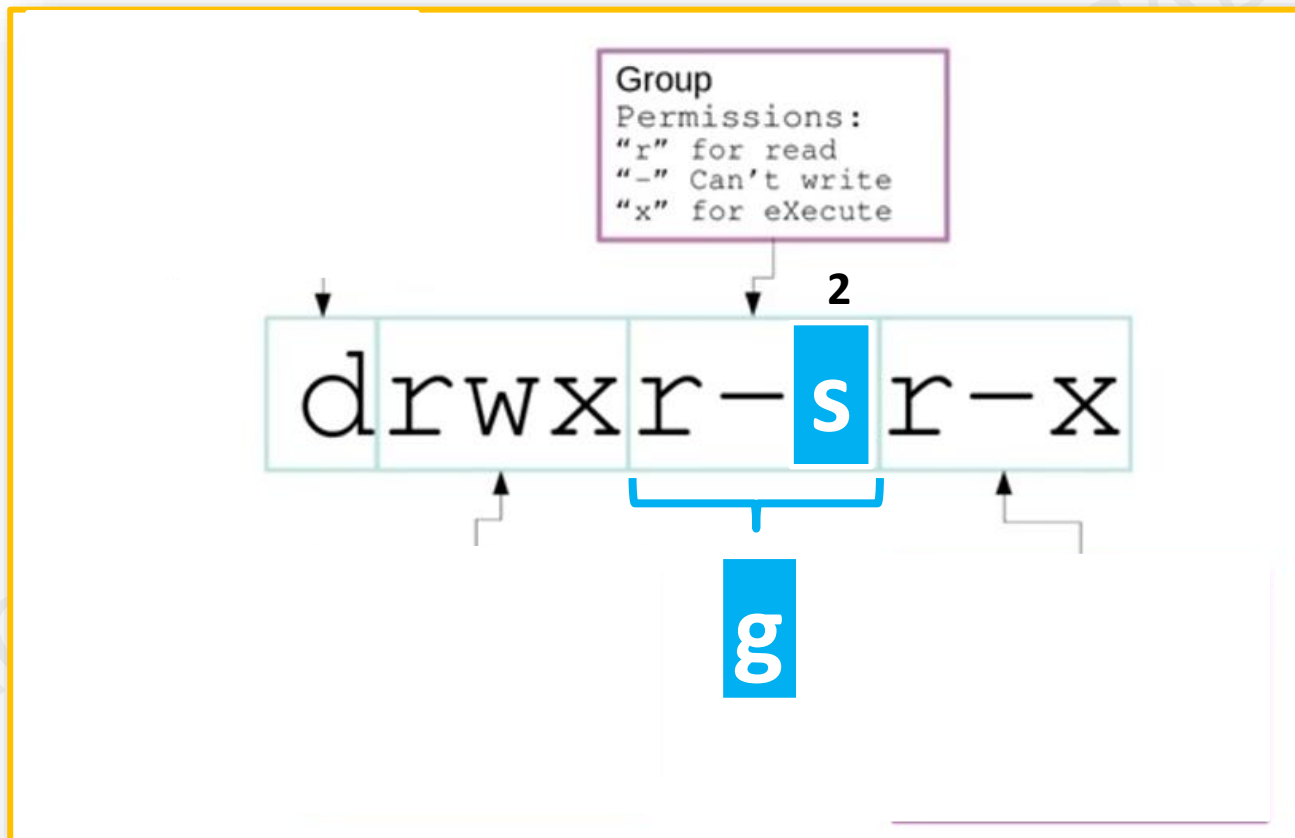
SGID

GID works on FILES and DIRECTORIES. **Mostly set on Directories!**

- If **SGID** is set on a directory, then all nested directories/files will be owned by the same group owner. This is often used to ease the sharing of files (files will be modifiable by all the users that are part of that group .
- The **FILE** which has the **setgid** bit set, when executed, instead of running with the privileges of the group of the user who started it, runs with those of the group which owns the file: in other words, the group ID of the process will be the same of that of the file.
- **Relative Mode:** #chmod g+s directory
- **Absolute Mode:** # chmod 2XXX directory

SPECIAL PERMISSIONS

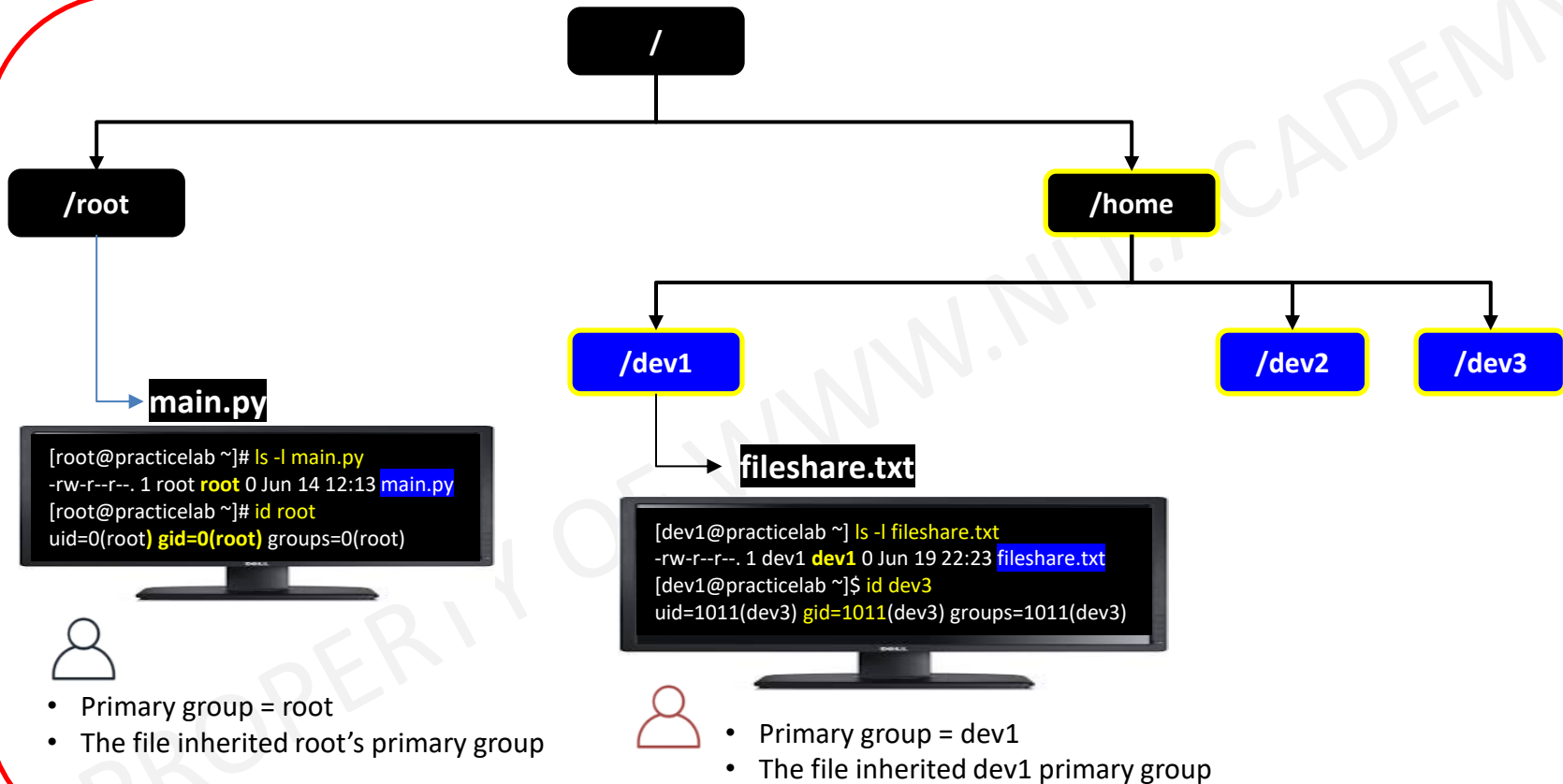
- Set UID = 4
- **Set SGID = 2**
- Sticky Bit = 1
- **NOTE:** always use STAT command to find details about a file or directory



FUNDAMENTAL CONCEPT

Normally, when a user creates a file, the file gets the user's **primary group**.

SELinux – Watchdog!



LINUX WITHOUT SETGID

Create a directory called share that belongs to a group called linuxteam

```
[root@practicelab ~]# mkdir -p /opt/share  
[root@Practicelab ~]# chown root:linuxteam /opt/share  
[root@Practicelab ~]# chmod 775 /opt/share  
drwxrwxr-x. 2 root linuxteam 6 Jun 20 00:08 /opt/share
```

SSH as dev3 and create a file in directory /share

```
> SSH dev3@192.168.1.xxx  
[dev3@practicelab ~]# cd /opt/shared  
[dev3@practicelab share]$ touch main.py  
touch: cannot touch 'main.py': Permission denied
```

Why is dev3 Blocked? Let us verify using namei -l command:

```
[dev3@practicelab share]$ namei -l /opt/share  
f: /opt/share  
dr-xr-xr-x root root /  
drwxr-xr-x root root opt  
drwxrwxr-x root linuxteam share
```

LINUX WITHOUT SETGID

Option 1

```
[root@practicelab ~]# usermod -aG linuxteam dev3
```

Note: “-aG vs -G”: means that -aG command appends linuxteam to other groups of dev3 user. Whereas -G will overwrite all other secondary groups of dev3 user and establish linuxteam as the ONLY secondary group

```
[root@practicelab share]# groups dev3
```

```
dev3 : dev3 linuxteam gitadmin
```

SSH as dev3 and let us try to create a file in /opt/share

```
> SSH dev3@192.168.1.xxx
```

```
[dev3@practicelab ~]# cd /opt/shared
```

```
[dev3@practicelab share]$ groups dev3
```

```
dev3 : dev3 linuxteam gitadmin
```

```
[dev3@practicelab share]$ touch newfile
```

```
touch: cannot touch 'newfile': Permission denied
```

Because dev3 is still using an old login session, and Linux only reads a user's group memberships at the exact moment they log in.

```
[dev3@practicelab share]$ newgrp linuxteam
```

(In real world please exit and SSH back in is the way!)

```
[dev3@practicelab share]$ touch newfile
```

```
[dev3@practicelab share]$ ls -l newfile
```

```
-rw-r--r--. 1 dev3 linuxteam 0 Jun 20 01:32 newfile
```

LINUX WITHOUT SETGID

Option 2

```
[root@practicelab ~]# chmod 777 /opt/share
```

This is not practical because now everyone in the entire company can access this /opt/share directory

```
> ssh dev2@192.168.1.97
```

```
[dev2@practicelab ~]$ ls -ld /opt/share
```

```
drwxrwxrwx root linuxteam share
```

Dev2 – infact Any user can now access this directory (folder):

```
[dev2@practicelab ~]$ cd /opt/share
```

```
[dev2@practicelab share]$ touch dev2file.sh
```

```
[dev2@practicelab share]$ ls -l
```

```
total 0
```

```
-rw-r--r--. 1 root root    0 Jun 20 01:10 app.jsx
```

```
-rw-r--r--. 1 dev2 dev2    0 Jun 20 01:38 dev2file.sh
```

```
-rw-r--r--. 1 dev3 linuxteam 0 Jun 20 01:32 newfile
```

```
drwxr-xr-x. 2 root root    6 Jun 20 01:10 project
```

MESSY Collaboration!

SETGID

#Let us SOLVE this problem using SetGID

```
[root@practicelab ~]# chmod 2775 /opt/share
```

```
# Or using symbolic mode: chmod g+s /opt/share
```

```
[dev3@practicelab share]$ namei -l /opt/share
```

```
f: /opt/share
```

```
dr-xr-xr-x root root /
```

```
drwxr-xr-x root root opt
```

```
drwxrwsr-x root linuxteam share
```

```
| | |
| | | — Others (r-x) -> No Write!
| | — Group (rws) -> Can Write!
| — Owner (rwx) -> Can Write!
```

Final Note!

```
[dev2@practicelab share]$ touch dev2.sh
```

```
touch: cannot touch 'dev2.sh': Permission denied
```

FUNDAMENTAL CONCEPT

WHY DID Dev2 get PERMISSION DENIED?

Here is the exact logic path for dev2:

- **Step 1:** Is dev2 the Owner? The **owner is root**. dev2 is not root. Move to Step 2
- **Step 2:** Is dev2 a member of the Group? The **group owner is linuxteam**. Since dev2's primary group is dev2 and they do not have linuxteam as a secondary group, they do not match the group. Move to Step 3.
- **Step 3:** Everyone Else (Others). Because dev2 failed the first two checks, dev2 falls into the **Others category**. The **permissions for Others are r-x (Read and Execute)**. There is **no w (write) bit here**. Because the "Others" category lacks write permissions, the kernel blocks the command and throws **Permission Denied**.
- What the **SGID** Bit is Doing Right Now: The **SGID bit (the s in drwxrwsr-x)** isn't a wall that blocks users; it's a **group inheritance rule** that only springs into action after a user successfully passes the 3-step filter above.
 - **Without SGID:** If an authorized user (like dev3) creates a file, the file's group owner becomes dev3.
 - **With SGID:** When dev3 creates a file, the SGID bit intercepts it and forces the file's group owner to be linuxteam. The SGID bit only dictates what happens to new files; it doesn't change who is allowed to enter or write to the folder.

WHAT IS STICKY BIT?

e, steered by safety™

The **sticky bit** works on directories only.

Let us learn about **/tmp** directory:

The **/tmp** is the directory for temporary files/directories. This directory has all the **rights on all the three levels** because all the users need to create/delete their temporary files.

Since all the users have “**write**” **permission** on this directory, then users can easily delete **any file in this directory**. The permissions of that file do not have any effect on deletion. So, how can we prevent users from deleting other users files in /tmp

Solution is Sticky Bit

Sticky bit when set on a directory means that anyone can create a file/directory in it, however, they can only delete their own files. Files owned by other users cannot be deleted.

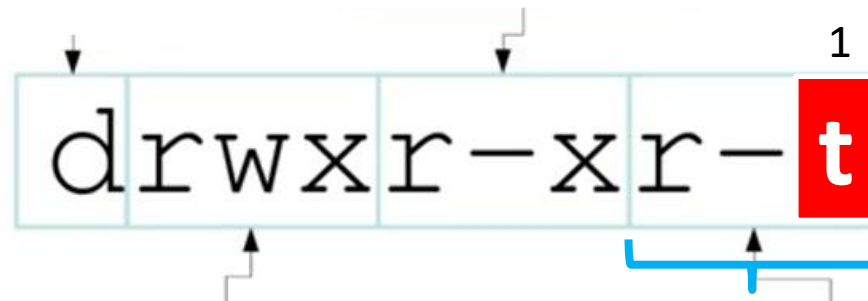
Absolute Mode: `chmod 1XXX directory`

Relative Mode : `chmod o+t directory`

```
[root@practicelab ~]# ls -ld /tmp
drwxrwxrwt. 10 root root 4096 Jun 20 01:51 /tmp
```

SPECIAL PERMISSIONS

- Set UID = 4
- Set GUID = 2
- **Sticky Bit = 1**
- **NOTE:** Most popularly used on /tmp folder



t

Other
Permissions:
"r" for read
"-" Can't write
"x" for eXecute

DON'T FORGET

By default, `userdel` retains the user's home directory. If you want to keep the home directory intact, omit the `-r` option.

NOTE: `#userdel` command will not remove users home directory

1. `#rmdir dir1`
2. `#rm -rf file1`

Option	Description
-f	Force removal of the user account, including home directory and mail spool, even if the user is logged in.
-r	Remove the user's home directory along with the account. Useful for a complete cleanup.
-v	verbose